

6.1 树的结构及基本操作

树是一类重要的非线性数据结构，是以分支关系定义的层次结构。

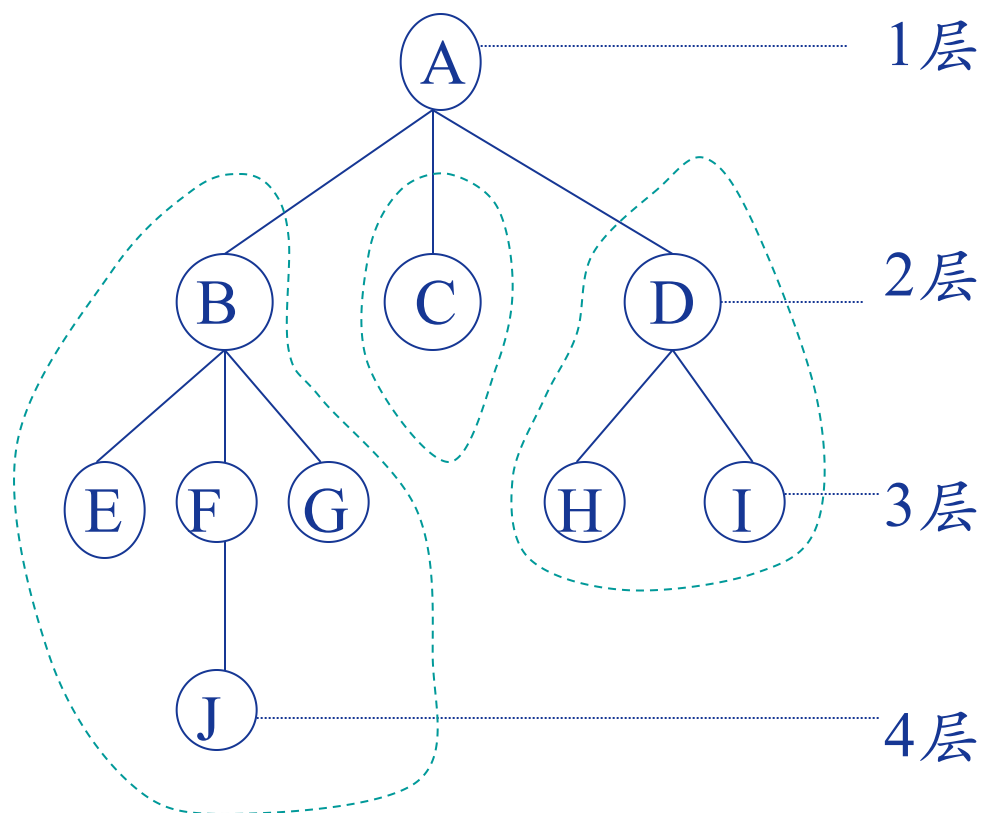
[树的定义]

树是由 $n(n \geq 0)$ 个结点组成的有限集合 T ，非空树满足：

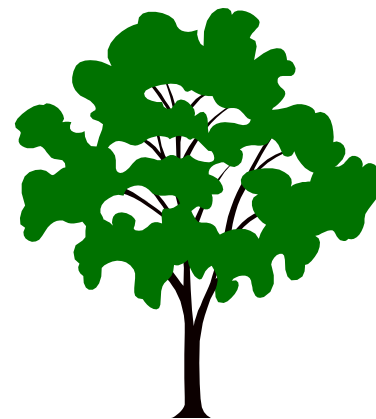
1) 有一个称之为根(root)的结点。

2) 除根以外的其余结点被分成 $m(0 \leq m < n)$ 个互不相交的集合 $T_1, T_2, \dots, T_m$ ，其中每一个集合本身又是一棵树，且称为根的子树。

[树的特点]



特点 除根结点外，
每个结点都**仅有一个**
前趋(父)结点。



[基本术语]

结点的度 结点拥有的子树数目。

叶子(终端)结点 度为0的结点。

分支(非终端)结点 度不为0的结点。

树的度 树的各结点度的最大值。

内部结点 除根结点之外的分支结点。

双亲与孩子(父与子)结点 结点的子树的根称为该结点的孩子；该结点称为孩子的双亲。

兄弟 属于同一双亲的孩子。

结点的祖先 从根到该结点所经分支上的所有结点。

结点的子孙 该结点为根的子树中的任一结点。

结点的层次 表示该结点在树中的相对位置。根为第一层，其它的结点依次下推；若某结点在第 L 层上，则其孩子在第 $L+1$ 层上。

堂兄弟 双亲在同一层的结点互为堂兄弟。

树的深(高)度 树中结点的最大层次。

有序树 树中各结点的子树从左至右是有次序的，不能互换。否则，称为**无序树**。

路径长度 从树中某结点 N_i 出发，能够“自上而下地”通过树中结点到达结点 N_j ，则称 N_i 到 N_j 存在一条路径，路径长度等于这两个结点之间的分支数。

树的路径长度 从根到每个结点的路径长度之和。

森林 是 $m(m \geq 0)$ 棵互不相交的树的集合。

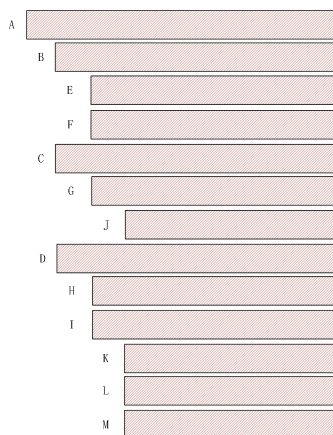
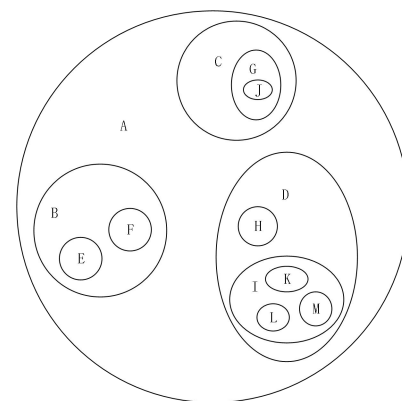
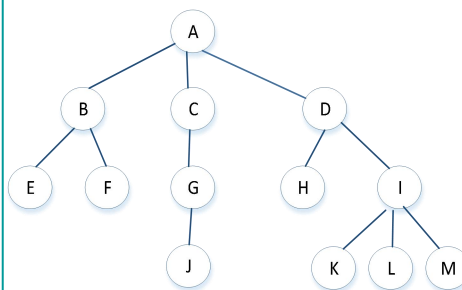
[树的逻辑表示]

①树形表示法：用圆圈表示一个结点，连线表示分支。

②文氏图表示法：每棵树对应一个圆圈，圆圈里包含根结点和子树圆圈，同一结点子树圆圈不相交。

③凹入表表示法：每棵树一个条形，子树的根对应较短的条形。同一结点的子树根条形长度一样。

④广义表(括号)表示法：每棵树对应一个以根为名的字符串，串名放左边，串是由括号里的各个子树对应的子串构成，子串之间用逗号隔开。



A(B(E,F),C(G(J)),D(H,I(K,L,M)))

[树的基本操作]

1)初始化操作	initiate(T)
2)求根函数	root(T) / root(x)
3)求双亲函数	parent(T,x)
4)求孩子结点函数	child(T,x,i)
5)求右兄弟函数	right_sibling(T,x)
6)建树函数	crt_tree(x,F)
7)插入子树操作	ins_child(y,i,x)
8)删除子树操作	del_child(x,i)
9)遍历操作	traverse(T)
10)清除结构操作	clear(T)

[树的性质]

性质①：树中结点数=所有结点度之和+1。

性质②：度为m的树第i层至多有 m^{i-1} 个结点，其中 $i \geq 1$ 。

说明：通常树根结点层次为1，依次类推。

性质③：高度为h，度为m的树，最多有 $(m^h - 1)/(m - 1)$ 个结点。

性质④：具有n个结点，度为m的树，最小高度为 $\lceil \log_m(n * (m - 1) + 1) \rceil$ 。

求解方法归纳

利用树的性质求解树中结点个数的一般方法：对于度为m的树，有：

$$n = n_0 + n_1 + \dots + n_m \quad \text{①} \quad \text{度之和} = n - 1 \quad \text{②}$$

$$\text{度之和} = n_1 + 2n_2 + \dots + m * n_m \quad \text{③}$$

联合①②③可得：

$$n = n_1 + 2n_2 + \dots + m * n_m + 1 = n_0 + n_1 + \dots + n_m$$

$$n_0 = n_2 + \dots + (m - 1) * n_m + 1$$

例 一棵度为3的树中有2个度为3的结点，1个度为2的结点，2个度为1的结点，则共有11个结点。

6.2 二叉树的定义与性质

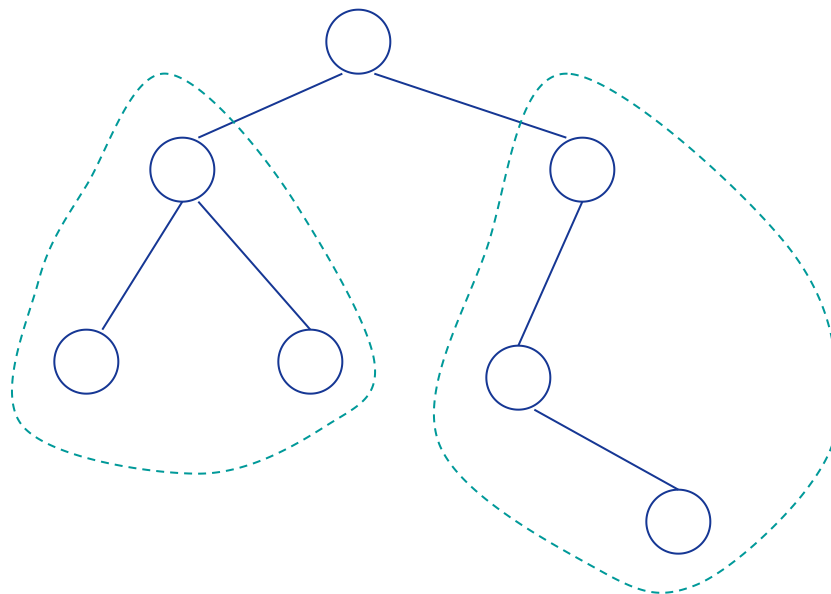
BUPT

[二叉树的概念]

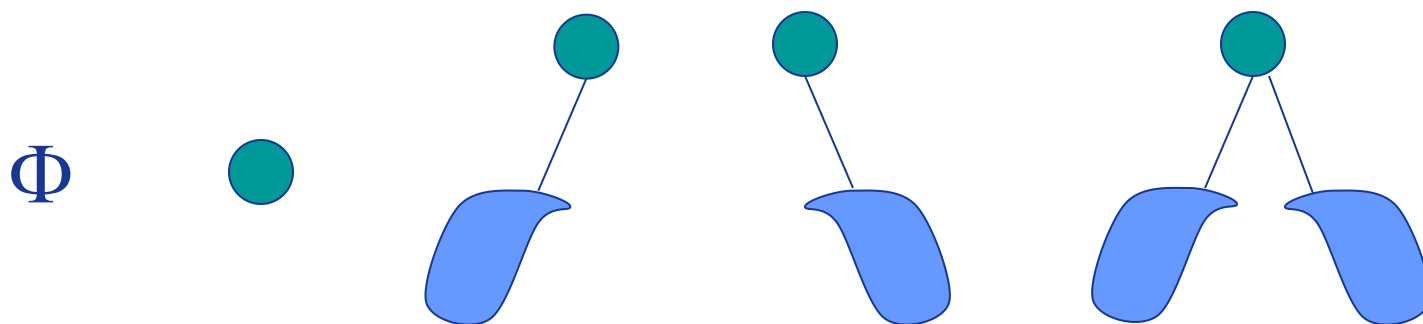
二叉树的定义 二叉树是 $n(n \geq 0)$ 个结点的有限集合，它或为**空树**($n=0$)，或由一个**根**结点和两棵互不相交的**左子树**和**右子树**的二叉树组成。

二叉树的特点

- ❖ 定义是递归的
- ❖ $0 \leq \text{结点的度} \leq 2$
- ❖ 是有序树



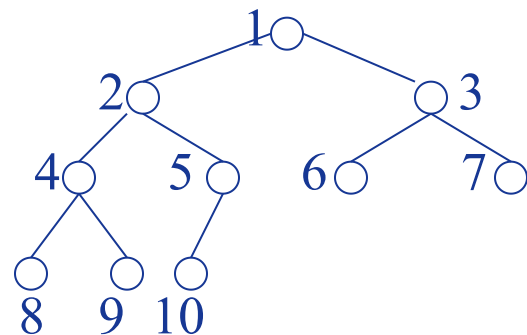
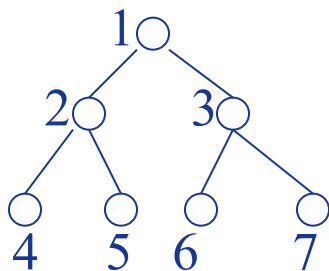
二叉树的五种基本形态



两种特殊的二叉树

满二叉树 每一层上的结点数都是最大结点数。

完全二叉树 只有最下面两层结点的度可小于2，而最下一层的叶结点集中在左边若干位置上。



[二叉树的性质]

性质1 二叉树的第 i 层上至多有 2^{i-1} ($i \geq 1$) 个结点。

性质2 深度为 k 的二叉树至多有 $2^k - 1$ 个结点 ($k \geq 1$)。

性质3 对任何一棵二叉树 T ，如果其终端结点数为 n_0 ，度为2的结点数为 n_2 ，则 $n_0 = n_2 + 1$ 。

性质4 具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$ 。

性质5 一棵具有 n 个结点的完全二叉树(又称顺序二叉树)，对其结点按层从上至下(每层从左至右)进行1至 n 的编号，则对任一结点 i ($1 \leq i \leq n$) 有：

(1) 若 $i > 1$ ，则 i 的双亲是 $\lfloor i/2 \rfloor$ ；若 $i = 1$ ，则 i 是根，无双亲。

(2) 若 $2i \leq n$ ，则 i 的左孩子是 $2i$ ；否则， i 无左孩子。

(3) 若 $2i + 1 \leq n$ ，则 i 的右孩子是 $2i + 1$ ；否则， i 无右孩子。

求解方法归纳

①利用二叉树的性质求结点个数的一般方法:

$$n = n_0 + n_1 + n_2$$

$$\text{度之和} = n - 1$$

$$\text{度之和} = n_1 + 2n_2$$

$$n_0 = n_2 + 1$$

②求解完成二叉树结点数的一般方法:

$$n \text{ 为偶数, } n_1 = 1; n_2 = \frac{n}{2} - 1; n_0 = \frac{n}{2}$$

$$n \text{ 为奇数, } n_1 = 0; n_2 = \frac{n-1}{2}; n_0 = \frac{n+1}{2}$$

③满二叉树高度 h 与 n , n_0 , n_1 , n_2 之间关系求解方法:

$$h = \log_2(n + 1); n_1 = 0; n = 2^h - 1; n_0 = 2^{h-1}; n_2 = 2^{h-1} - 1。$$

例1 一棵二叉树中有7个叶子, 5个度为1的结点, 则共有18个结点。

例2 一颗完全二叉树共有1001个结点, 则叶子有501个。

例3 一颗满二叉树共有64个叶子, 则树高为7。

6.3 二叉树的存储结构

BUPT

顺序存储结构

[完全二叉树：按完全二叉树编号存放]

A	B	C		D		E
1	2	3		6		13

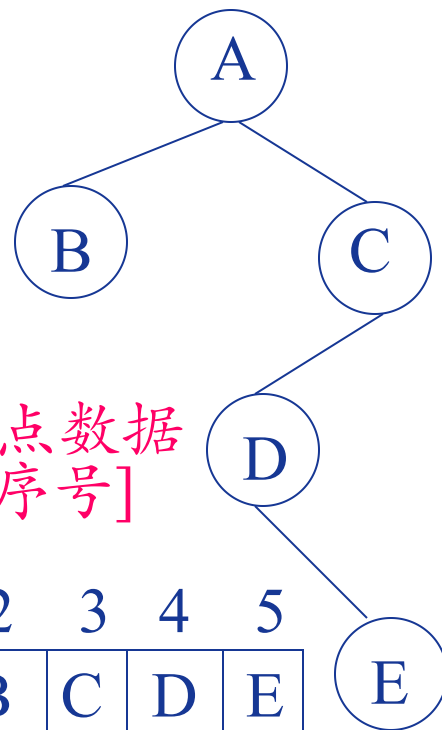
[三元组：存储结点数据和

左、右孩子在向量中的序号]

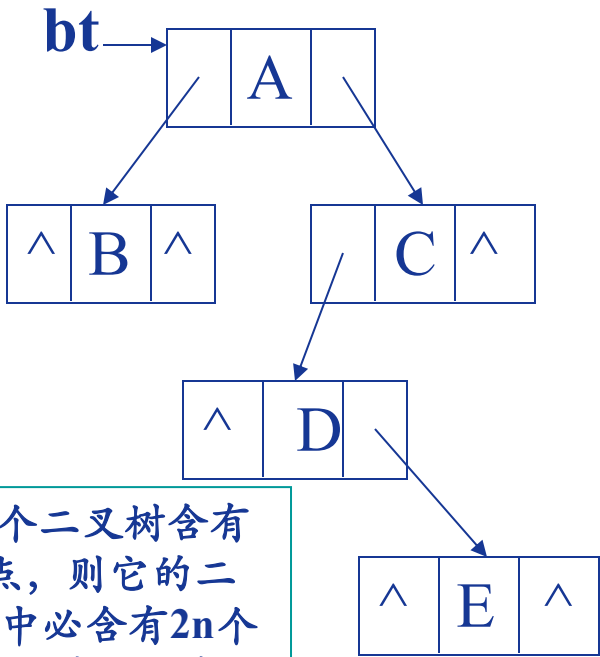
	0	1	2	3	4
data	A	B	C	D	E
lc	1	-1	3	-1	-1
rc	2	-1	-1	4	-1

[双亲：存储结点数据和其父结点的序号]

	1	2	3	4	5
data	A	B	C	D	E
parent	0	1	-1	3	-4

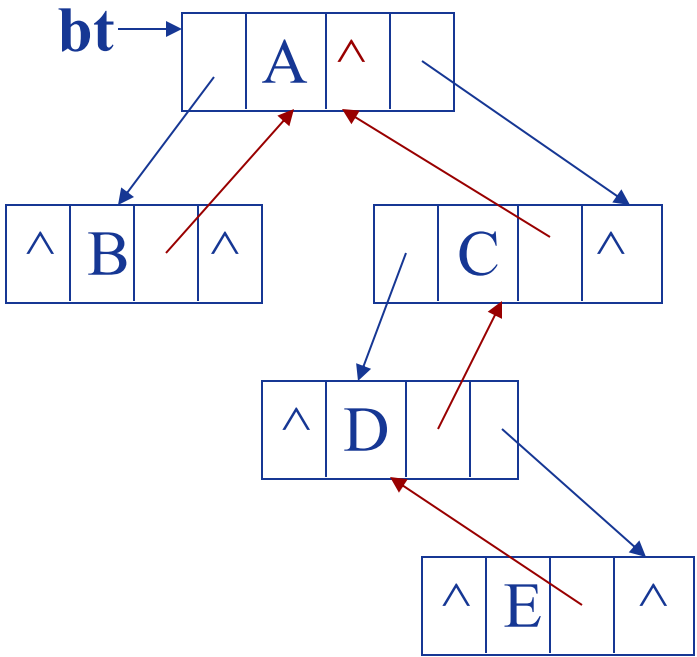


[二叉链表]



结论: 若一个二叉树含有 n 个结点, 则它的二叉链表中必含有 $2n$ 个指针域, 其中必有 $n+1$ 个空的链域。

[三叉链表]
又称带双亲的二叉链表



二叉链表的类型定义

```
struct btnode
{
    struct btnode *lc;
    struct btnode *rc;
    elemtype data;
};
typedef struct btnode *PtrToNode;
typedef PtrToNode BiTree;
```

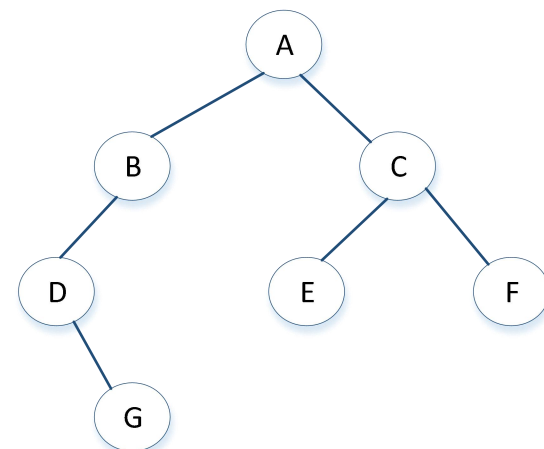
三叉链表的类型定义

```
typedef
struct btnode
{
    struct btnode *lc;
    struct btnode *rc;
    struct btnode *parent;
    elemtype data;
} *BiTree;
```

【示例】：用串A(B(D(,G),C(E,F)))可以表示如右所示一颗二叉树，称之为二叉树的广义表表示法，请按广义表表示法，创建二叉链表存储的二叉树。

思路：利用栈来保存双亲结点，k为标志。用ch扫描字符串，分以下几种情况：

- ①若ch='('，则将刚创建的结点作为双亲进栈，并置标志k=1，表示其后创建的是这个栈顶结点的左孩子；
- ②若ch=')'，表示栈顶结点的左右孩子都处理完毕，栈顶结点退栈；
- ③若ch=','表示其后创建的是栈顶结点的右孩子，置标志k=2；
- ④其他情况，表示要创建一个结点，并根据k值建立它与栈顶结点之间的关系；若k=1;表示该结点为栈顶结点的左孩子，则栈顶结点的lc要指向该结点；若k=2;表示该结点为栈顶结点的右孩子，则栈顶结点的rc要指向该结点。



```

void CreateTree(BiTree &T, char *str)
{
    BiTree St[MAXSIZE]; BiTree p = NULL;
    int top = -1, k, j = 0; char ch = str[j];
    T = NULL;
    while (ch != '\0')
    {
        switch(ch)
        {
            case '(': top++; St[top] = p; k=1; break; \为左子
            case ')': top--; break;
            case ',': k=2; break; \为右子
            default: p = (BiTree)malloc(sizeof(struct btnode)); ... \省略了出错处理;
                p->data = ch; p->lc = p->rc = NULL;
                if (b==NULL) b = p;
                else
                {
                    if (k == 1) St[top]->lc = p;
                    if (k == 2) St[top]->rc = p;
                }
        }
        ch = str[++j];
    }
}

```

6.4 二叉树的遍历

BUPT

[遍历的目的] 非线性结构 $\Rightarrow$ 线性结构

[遍历的概念] 指按某条搜索路线走遍二叉树的每个结点，使得树中每个结点都被访问一次，且仅被访问一次。

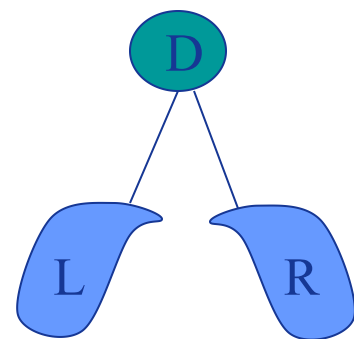
[典型的遍历方法]

先(根)序遍历 DLR

中(根)序遍历 LDR

后(根)序遍历 LRD

层序遍历



上述逆序方式很少使用

[算法描述]

先序遍历非空二叉树:

- ①访问根;
- ②先序遍历左子树;
- ③先序遍历右子树。

中序遍历非空二叉树:

- ①中序遍历左子树;
- ②访问根;
- ③中序遍历右子树。

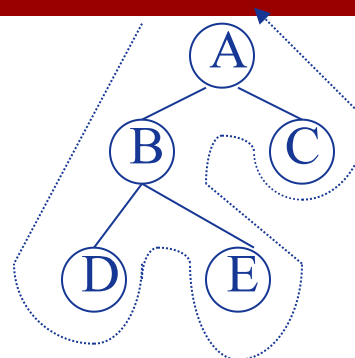
后序遍历非空二叉树:

- ①后序遍历左子树;
- ②后序遍历右子树
- ③访问根;

先序遍历: ABDEC

中序遍历: DBEAC

后序遍历: DEBCA



[利用遍历结果确定二叉树]

先序序列+中序序列

中序序列+后序序列

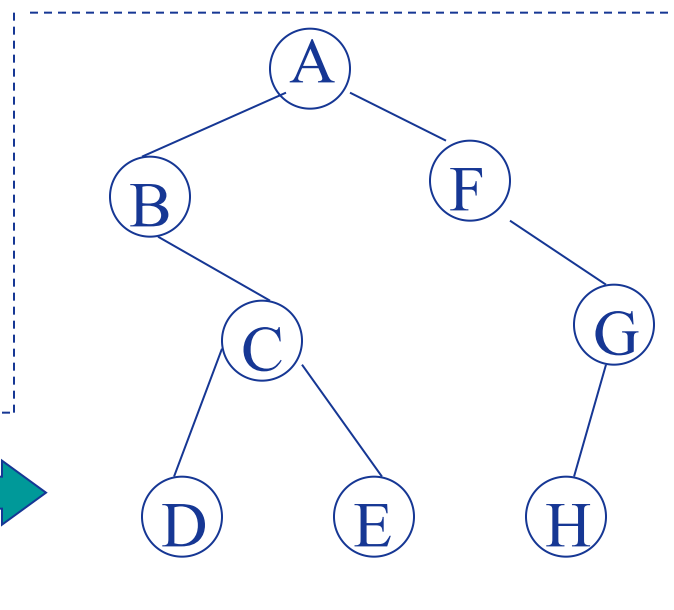
先序序列+后序序列 ×

思考: 层序+先序/中序/后序

能否确定? 如何做?

先序序列: ABCDEFGH

中序序列: BDCEAFHG

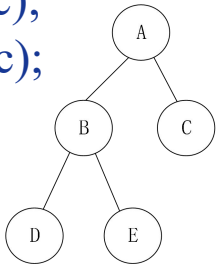


[先序遍历算法]

[递归算法]

void Preorder (BiTree T)

```
{  
    if (T)  
    {  
        Printdata(T->data);  
        L1: Preorder(T->lc);  
        L2: Preorder(T->rc);  
        L3:}  
    }
```



①
L2 A

先序: A

②
L2 B
L2 A

先序: AB

③
L2 D
L2 B
L2 A

先序: ABD

D左子空递归返回,退栈根为D,执行L2

④
L2 B
L2 A

先序: ABD

⑤
L3 D
L2 B
L2 A

先序: ABD

D右子空递归返回,退栈根为D,执行L3

⑥
L2 B
L2 A

先序: ABD

L3程序尾,递归返回,退栈根为B,执行L2

⑦
L2 A

先序: ABD

⑧
L3 B
L2 A

先序: ABD

⑨
L2 E
L3 B
L2 A

先序: ABDE

E左子空递归返回,退栈根为E,执行L2

⑩
L3 B
L2 A

先序: ABDE

⑪
L3 E
L3 B
L2 A

先序: ABDE

E右子空递归返回,退栈根为E,执行L3

⑫
L3 B
L2 A

先序: ABDE

L3程序尾,递归返回,退栈根为B,执行L3

⑬
L2 A

先序: ABDE

L3程序尾,递归返回,退栈根为A,执行L2

⑭

先序: ABDE

⑮
L3 A

先序: ABDE

⑯
L2 C
L3 A

先序: ABDEC

C左子空递归返回,退栈根为C,执行L2

⑰
L3 A

先序: ABDEC

⑱
L3 C
L3 A

先序: ABDEC

C右子空递归返回,退栈根为C,执行L3

⑲
L3 A

先序: ABDEC

L3程序尾,递归返回,退栈根为A,执行L3

⑳

先序: ABDEC

L3程序尾,递归结束,返回调用程序

㉑

先序: ABDEC

[消除尾递归的递归算法]

```
void Preorder3 (BiTree T)
```

```
{
```

```
    while (T)
```

```
    {
```

```
        Printdata(T->data);
```

```
        Preorder3(T->lc);
```

```
        T = T->rc;
```

```
    }
```

```
}
```

插入栈初始化语句

改写递归，当前工作记录入栈，参数重新赋值，转被调用过程

递归返回，栈顶工作记录出栈，按栈顶参数值从返回地址开始继续往下执行。

[非递归算法]

```

void Preorder4 (BiTree T)
{
    inistack(s);
    BiTree p =T;
    push(s, NULL); //空元素入栈
    while (p )
    {
        Printdata(p ->data);
        if (p->rc)
            push(s, p->rc);
        if ( p->lc)
            p = p->lc;
        else
            p = pop(s);
    }
}

```

[非递归算法2]

```

void Preorder5 (BiTree T)
{
    inistack(s);
    BiTree p =T;
    while (!empty(s) || p)
    {
        if (p)
        {
            Printdata(p ->data);
            push(s,p);
            p =p->lc;
        }
        else
        {
            pop(s,p)
            p = p->rc;
        }
    }
}

```

[中序遍历算法]

```
void Inorder (BiTree T)
{
    if (T)
    {
        Inorder(T->lc);
        Printdata(T->data);
        Inorder(T->rc);
    }
}
```

[后序遍历算法]

```
void Postorder(BiTree T)
{
    if (T)
    {
        Postorder(T->lc);
        Postorder(T->rc);
        Printdata(T->data);
    }
}
```

[按层次遍历算法]

```
void Levelorder (BiTree T);  
{  
    iniqueue( q);  
    if (T) enqueue( q, T);  
    while (!empty( q))  
    {  
        x= dequeue( q);  
        Printdata(x->data);  
        if (x->lc) enqueue( q, x->lc );  
        if (x->rc) enqueue( q, x->rc );  
    }  
}
```

三种递归算法分析

Order(BiTree T)

{

L1: Printdata(T->data); //先序访问

L2: Order(T->lc); //入栈

L3: Printdata(T->data); //出栈中序访问

L4: Order(T->rc); //入栈

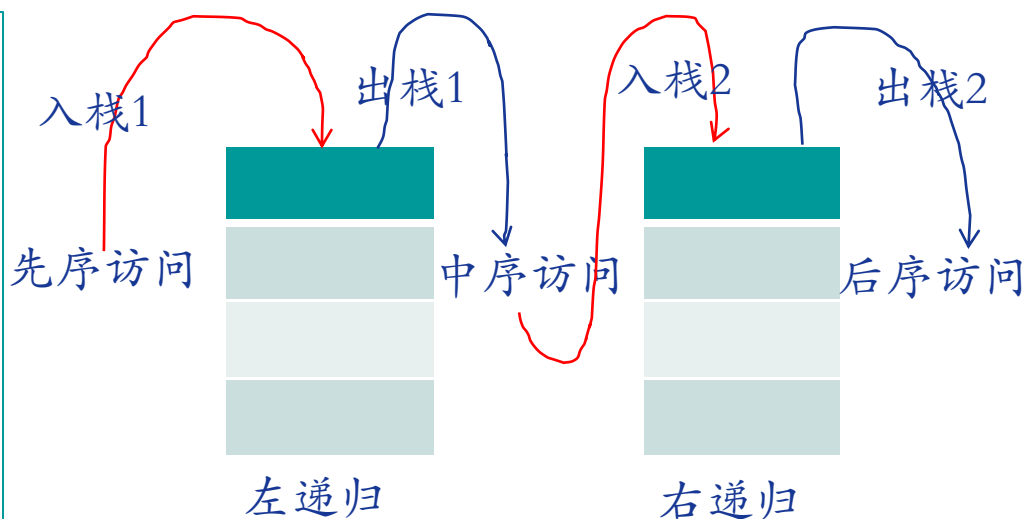
L5: Printdata(T->data); //出栈，后序访问

}

L1+L2+L4:先序

L2+L3+L4:中序

L2+L4+L5:后序

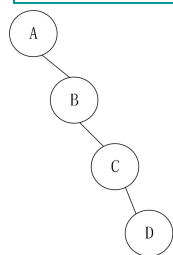


入栈1的顺序为先序序列
出栈1的顺序为中序序列
入栈2的顺序为中序序列
出栈2的顺序为后序序列

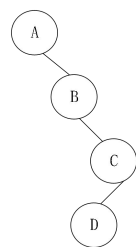
重要结论 树的计数

- ①若把**先**序序列看成入栈序列，则**中**序序列对应着合法的出栈序列。
- ②若把**中**序序列看成入栈序列，则**后**序序列对应着合法的出栈序列。
- ③若已知含n个结点的二叉树的先序序列，则可能的中序序列有 $(2n)!/[(n+1)!n!]$ ，对应的树型也有 $(2n)!/[(n+1)!n!]$ 。
- ④若已知含n个结点的二叉树的中序序列，则可能的后序序列有 $(2n)!/[(n+1)!n!]$ ，对应的树型也有 $(2n)!/[(n+1)!n!]$ 。

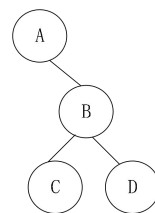
例1 若二叉树的先序序列为ABCD,则可能的二叉树的有多少种? **14种**



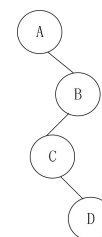
中序ABCD



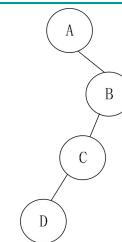
中序ABDC



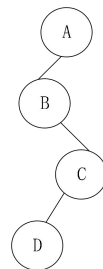
中序ACBD



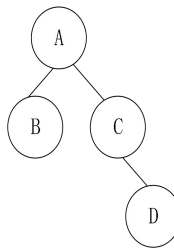
中序ACDB



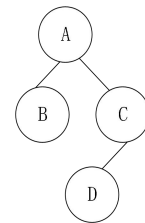
中序ADCB



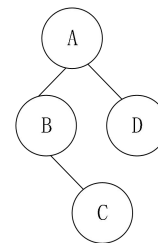
中序BDCA



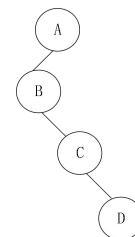
中序BACD



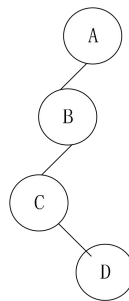
中序BADC



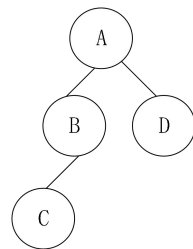
中序BCAD



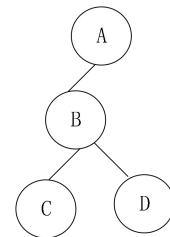
中序BCDA



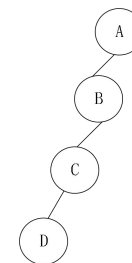
中序CDBA



中序CBAD

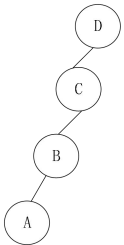


中序CBDA

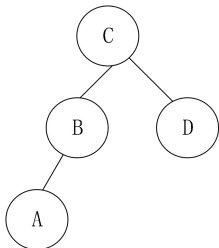


中序DCBA

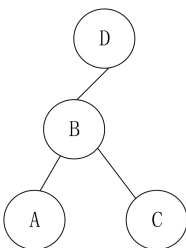
例2 若二叉树的中序序列为ABCD,则可能的二叉树的有多少种? **14种**



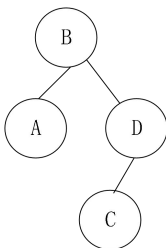
后序ABCD



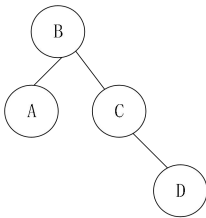
后序ABDC



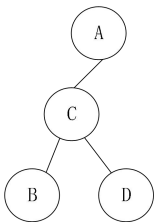
后序ACBD



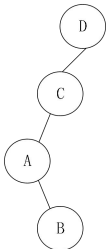
后序ACDB



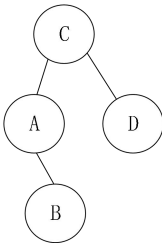
后序ADCB



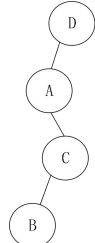
后序BDCA



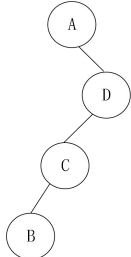
后序BACD



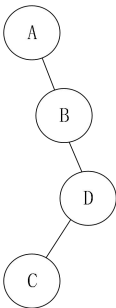
后序BADC



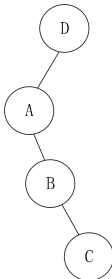
后序BCAD



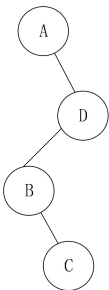
后序BCDA



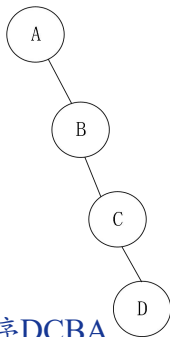
后序CDBA



后序CBAD



后序CBDA



后序DCBA

对用二叉链表表示的二叉树，设计一个算法，求其后序遍历的第一个结点。

```

bitptr firstnode(BiTree root)
{
    if (!root) return NULL;    // 空树不存在此结点
    while (root->lc || root->rc) // 尚未找到叶子结点
    {
        if (root->lc)
            root = root->lc;
        // 若有左子树，则沿左分枝向下查找最左下的叶子结点
    }
    else
        root = root->rc;
        // 若无左子树，则沿右分枝向下查找最左下的叶子结点
    }
    return root;                // 返回后序遍历的第一个结点
}

```

算法思想：

- (1) 若树根有左子树，则树根的左子树上最左下的叶子结点即为所求；
- (2) 若树根无左子树，仅有右子树，则树根右子树上最左下的叶子结点即为所求。

以下程序是二叉链表树中序遍历的非递归算法，请填空使之完善。

```

void vst(BiTree bt)                /*bt为根结点的指针*/
{
    bitree p; p=bt; initstack(s); /*初始化栈s为空栈*/
    while(p || !empty(s))          /*栈s不为空*/
        if(p)
        {
            push (s,p);
            (1)____;
        } /*P入栈*/
        else
        {
            p=pop(s);
            printf("%c",p->data);
            (2)____;
        } /*栈顶元素出栈*/
    }
}

```

(1) p=p->lc // 沿左子树向下
(2) p=p->rc

试找出满足下列条件的二叉树

- 1) 先序序列与后序序列相同
- 2) 中序序列与后序序列相同
- 3) 先序序列与中序序列相同
- 4) 中序序列与层次遍历序列相同

解答:

若先序序列与后序序列相同, 则或为空树, 或为只有根结点的二叉树

若中序序列与后序序列相同, 则或为空树, 或为任一结点至多只有左子树的二叉树.

若先序序列与中序序列相同, 则或为空树, 或为任一结点至多只有右子树的二叉树.

若中序序列与层次遍历序列相同, 则或为空树, 或为任一结点至多只有右子树的二叉树

[在二叉树中查找结点值为x的结点]

void **pre_find**(BiTree bt, ElemType x, BiTree &q)// 用q返回

{ //F是全局bool型变量，初值为FALSE

if (bt && **F**==FALSE)

{

if (bt->data==x)

{ q=bt; F=TRUE;}

else

{ **pre_find**(bt->lc, x, q);

pre_find(bt->rc, x, q)

}

}

}

```
{.....
q=NULL;
bool F=FALSE;
pre_find(bt,x,q);
.....
}
```

[求二叉树中每个结点所处的层次]

```
void pre_level(BiTree p, int level)
```

```
{  
    if (p)  
    {  
        write(p->data, level); //假设write是输出结点层次  
        pre_level(p->lc, level+1);  
        pre_level(p->rc, level+1);  
    }  
}
```

```
{.....  
    pre_level(bt, 1);  
    .....  
}
```

[求二叉树的高度]

```
void pre_height(BiTree p, int level)
```

```
{  if (p)
    {  if (h<level)  h=level;
        pre_height(p->lc, level+1);
        pre_height(p->rc, level+1);
    }
}
```

```
{.....
  h=0;
  pre_height(bt, 1);
  .....
}
```

```
void post_height(BiTree bt, int &h)
```

```
{  if (bt==NULL) h=0
    else
    {  post_height(bt->lc, h1);
        post_height(bt->rc, h2);
        h=1+max(h1, h2);
    }
}
```

```
{.....
  post_height(bt, h);
  .....
}
```

[复制一颗二叉树]

```
void pre_copy(BiTree bt; BiTree &q)
{
    if (bt)
    {
        new(q);
        q->data= bt->data;
        pre_copy(bt->lc, q->lc);
        pre_copy(bt->rc, q->rc);
    }
    else
        q=NULL;
}
```

```
{.....
    pre_copy(bt, q);
    .....
}
```

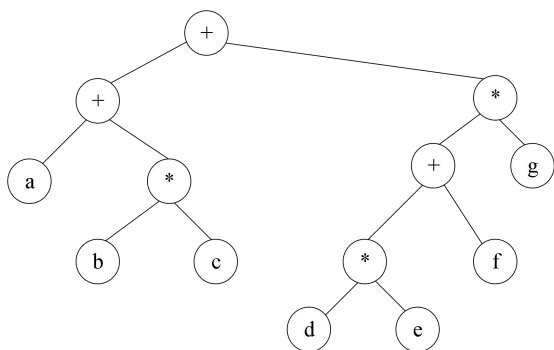
已知一棵二叉树按顺序方式存储在数组A[0..n]中（A[0]不存放结点信息），设计算法求下标分别为i和j的两个结点的最近公共祖先结点。

```
typedef etype completetree [ maxnodenum ];  
int forefather( completetree T, int i, int j )  
{  
    while ( i != j )  
        if ( i > j )  
            i = i / 2; // 下标为i的结点的双亲结点的下标  
        else  
            j = j / 2; // 下标为j的结点的双亲结点的下标  
    return i; // 返回最近祖先结点的下标  
}
```

6.5 算术表达式二叉树

BUPT

[定义] 表达式树 (Expression Tree) 树叶为操作数, 比如常量或者变量, 其他节点为操作符。在简单算术表达式中操作符均为二元的, 因此简单算术表达式是一颗二叉树。可通过递归计算左右子树得到的值, 再应用在根处的操作符, 进而计算出表达式的值。



中序 (中缀表达式) : ①递归产生一个带括号的左表达式; ②打印根结点的运算符; ③递归产生一个带括号的右表达式。

左子树: $a+(b*c)$;

右子树: $((d*e)+f)*g$

中缀\标准表达式: $(a+(b*c))+(((d*e)+f)*g)$

后序 (后缀\逆波兰表达式) : ①递归打印左子树; ②递归打印右子树; ③打印运算符。

后缀表达式: $abc*+de*f+g*+$

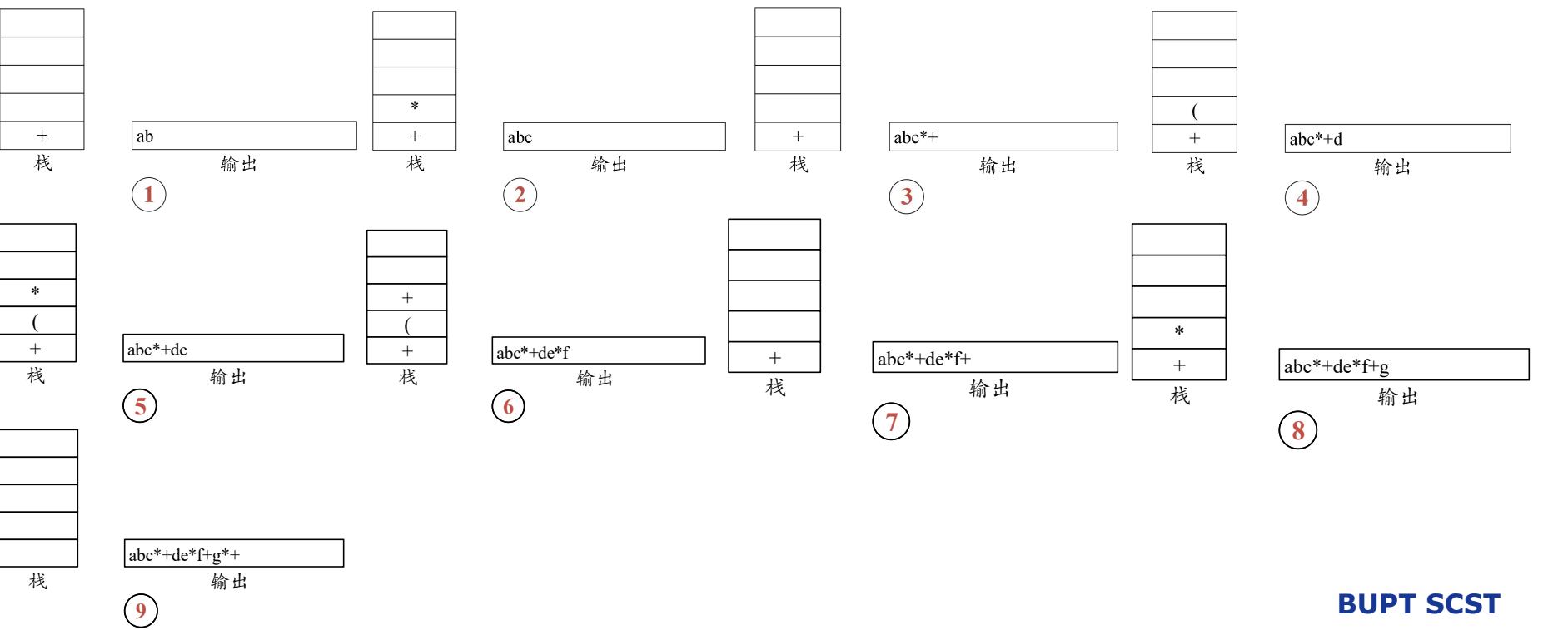
先序 (前缀\波兰表达式) : ①打印运算符; ②递归打印左子树; ③递归打印右子树。 **不常用**

前缀表达式: $++a*bc*+*defg$

[例] 将中缀表达式转为后缀表达式。

$a+b*c+(d*e+f)*g$ -----> $abc*+de*f+g*+$

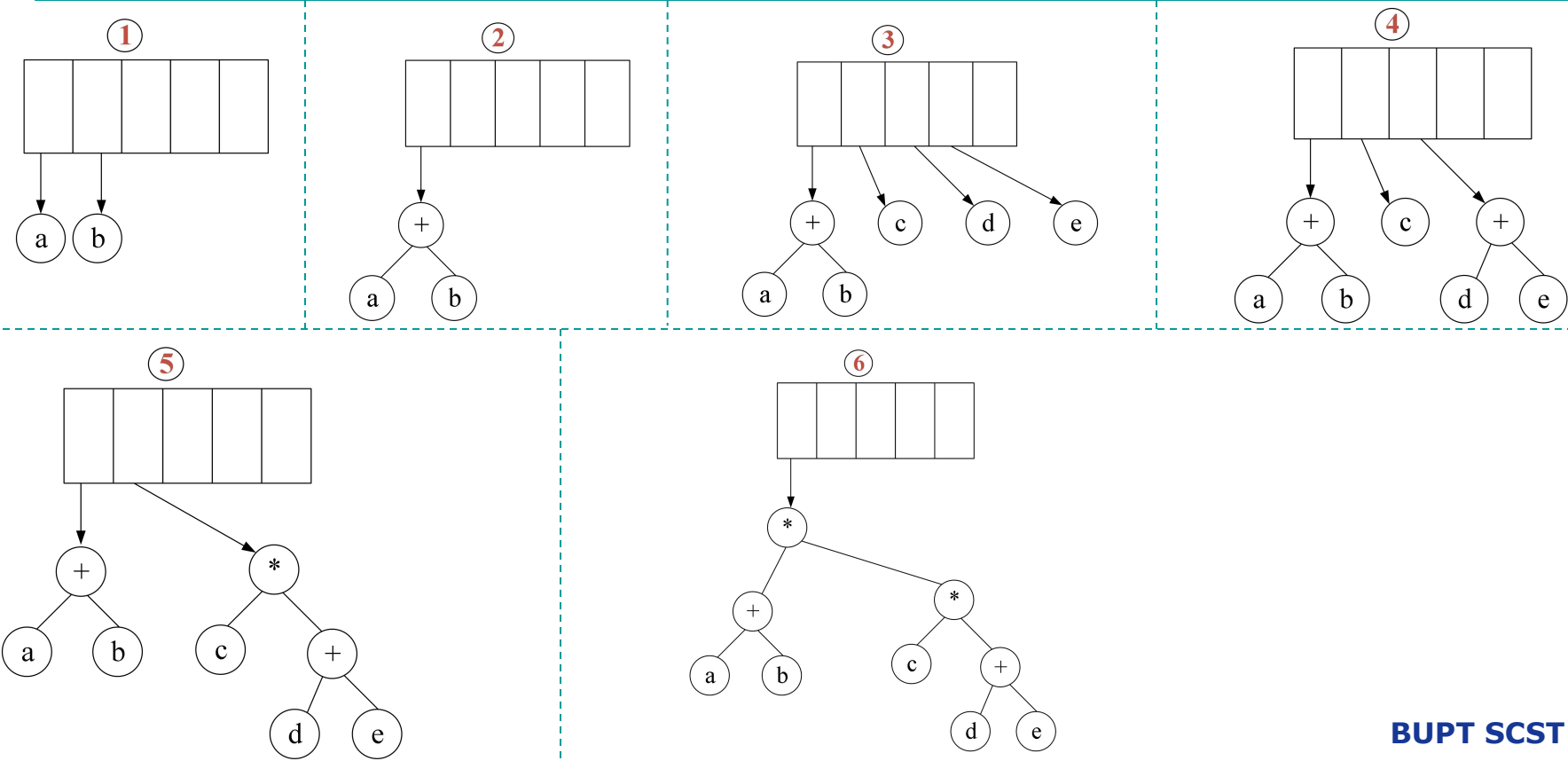
思路：①读到操作数直接输出；②读到运算符与栈顶运算符比较，如栈顶优先级低，新运算符入栈，否则栈顶运算符弹出，直到栈顶运算符优先级更低；③读到左括号入栈；④读到右括号栈中元素依次出栈，直到遇到左括号和它匹配，左括号出栈但不输出，除非是处理右括号，否则左括号绝不出栈；⑤输入空，栈中剩余部分全部输出。



[例] 将后缀表达式转为表达式树。

思路：①遇操作数，建立单结点树，并将其指针入栈；②遇操作符，栈中连续弹出指向两棵二叉树T1（先弹出）和T2的指针，并形成一颗新二叉树，根为操作符，它的左右子树分别为T1 和T2，将新树的指针入栈。

如： $ab+cde+**$



6.6 线索二叉树

BUPT

[引入线索二叉树的意义]

利用二叉链表的空指针域，建立指向该结点的前驱/后继(某种遍历方式下)结点的指针，方便二叉树的线性化使用。(n个结点的二叉树含有n+1空指针域)

[线索二叉树的存储结构]

lc	ltag	data	rtag	rc
----	------	------	------	----

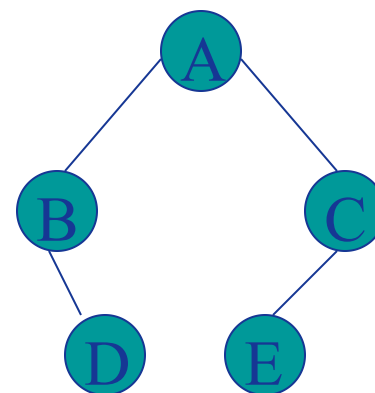
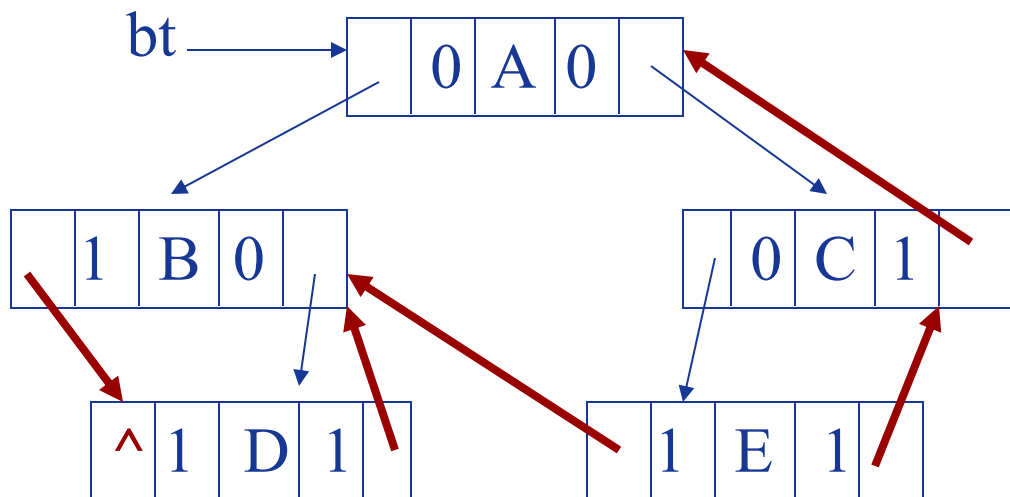
左特征位 ltag = $\begin{cases} 0 & \text{lc域指示结点的左孩子} \\ 1 & \text{lc域指示结点的前驱结点} \end{cases}$

右特征位 rtag = $\begin{cases} 0 & \text{rc域指示结点的右孩子} \\ 1 & \text{rc域指示结点的后继结点} \end{cases}$

后序线索二叉树

DBECA

[例]



[类型定义]

```
typedef enum PointerTag {Link, Thread};
typedef struct BiThrNode {
    TElemType data;
    struct BiThrNode *lc, *rc;
    PointerTag lTag, rTag;
} BiThrNode, BiThrTree;
```

[查找结点p->在指定次序下的前驱 / 后继结点]

❖ 中序线索二叉树

■ p的后继

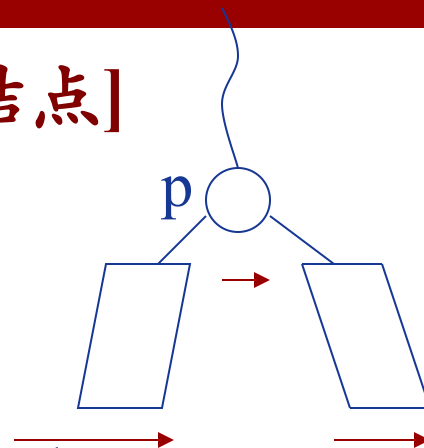
若 $p \rightarrow rtag = 1$ ，则 $p \rightarrow rc$ 即为所求；

若 $p \rightarrow rtag = 0$ ，则从其右子沿着左链走到 $ltag = 1$ 的那个结点就是。

■ p的前驱

若 $p \rightarrow ltag = 1$ ，则 $p \rightarrow lc$ 即为所求；

若 $p \rightarrow ltag = 0$ ，则从其左子沿着右链走到 $rtag = 1$ 的那个结点就是。



❖ 后序线索二叉树

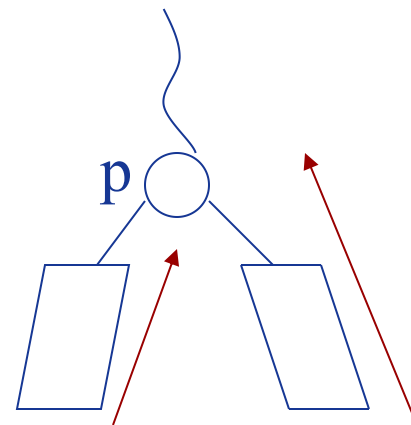
■ p的前驱

若 $p \rightarrow ltag = 1$ ，则 $p \rightarrow lc$ 即为所求；

若 $p \rightarrow ltag = 0$ ，

则若 $p \rightarrow rtag = 0$ ，则 $p \rightarrow rc$ 即为所求

若 $p \rightarrow rtag = 1$ ，则 $p \rightarrow lc$ 即为所求



■ p的后继

与双亲结点有关，因二叉链表中没有指向双亲结点的指针，就可能需通过二叉树的后序遍历才可确定，因而后序线索二叉树在此问题上并不比普通二叉树有效。

❖ 先序线索二叉树

与后序线索二叉树对偶

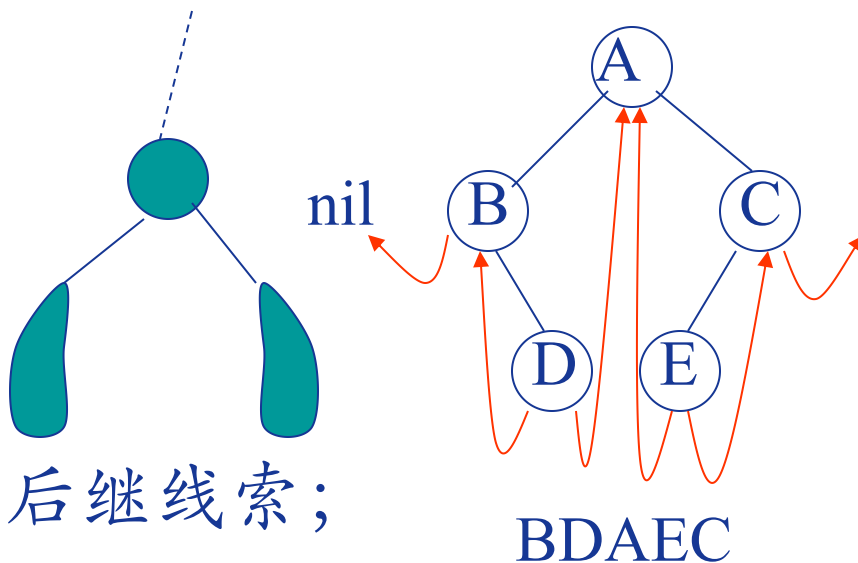
二叉树中序线索化

[算法思想] 利用中序遍历算法

[算法步骤]

若二叉树T非空，

- 1) 左子树线索化;
- 2) 访问当前结点p:
 - 2.1) 置lTag和rTag;
 - 2.2) 建立p的前趋pre的后继线索;
 - 2.3) 建立p的前趋线索;
 - 2.4) 将p作为下一个被访问结点的前趋结点
- 3) 右子树线索化;



[算法描述]

```

void inthread(BiThrTree p)
    if (p!=NULL)
    {
        inthread(p->lc);
        if (p->lc ==NULL) { p->ltag=1; p->lc=pre};
        if (pre !=NULL)
            if (pre->rc==NULL)
            {
                pre->rtag=1;
                pre->rc=p;
            }
        pre=p;
        inthread(p->rc)
    }

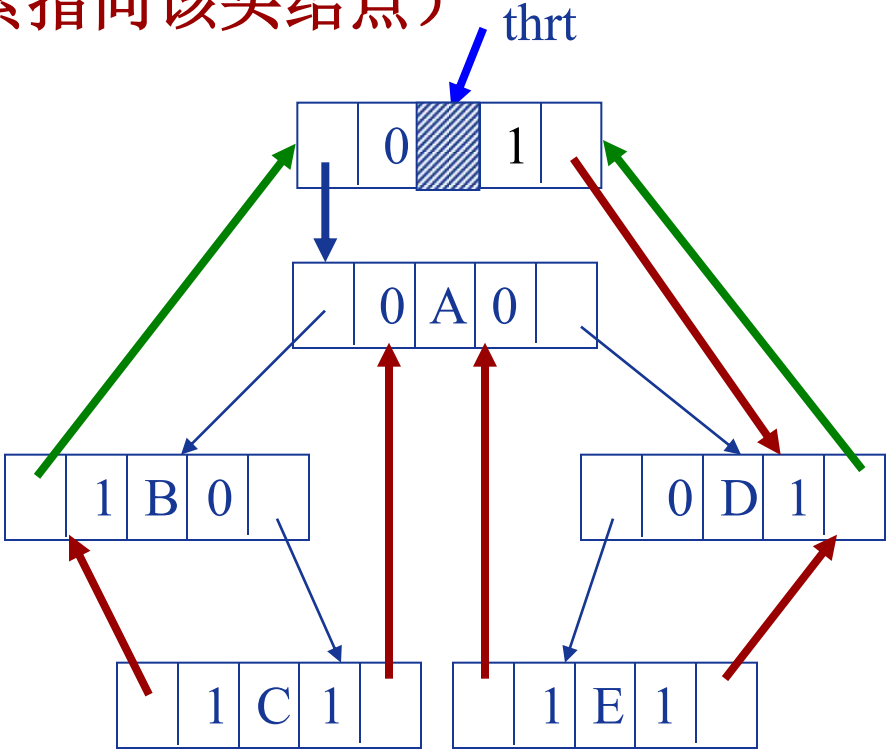
```

```

{.....
    pre=NULL; {全局变量}
    inthread(bt);
    .....
}

```

为使用方便，可以为线索二叉树建立头结点，构成双向
线索链表（空线索指向该头结点）



中序序列：BCAED
带头结点的中序线索二叉树

[线索二叉树的更新操作]

[问题] 在线索二叉树上插入或删除一棵子树或者结点时，指针和线索都需作相应的改动。

[解决方法]

- 1) 还原为普通二叉树，再重新将其线索化。
- 2) 讨论可能出现的各种情况，寻找规律。

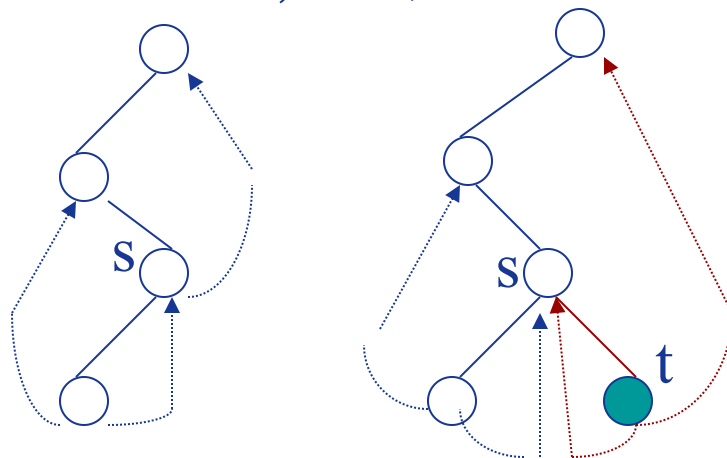
[示例1] 在中序线索二叉树中，插入结点t，使其成为结点s的右孩子。

❖ s无右孩子

s的后继成为t的后继；

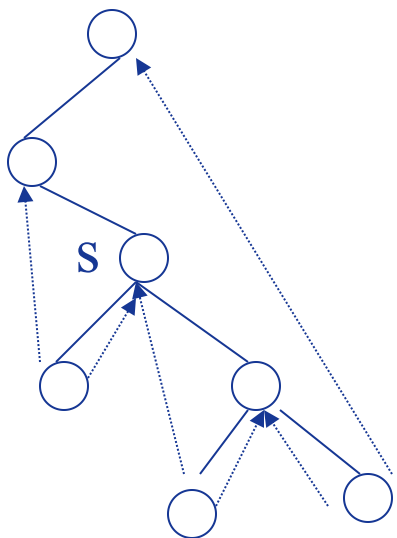
s成为t的前驱；

t成为s的右孩子

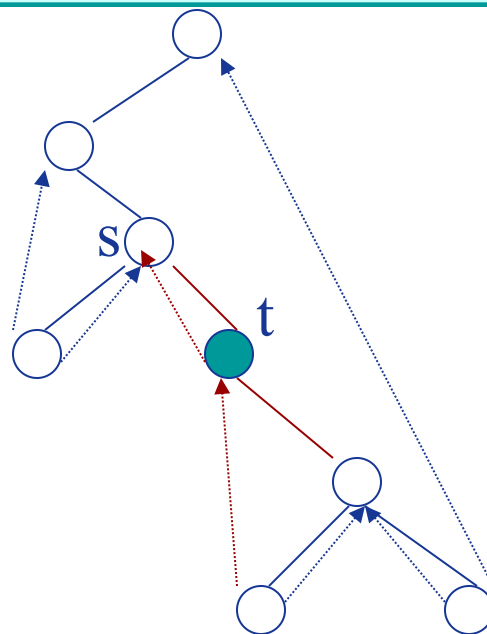


❖ s有右孩子

s的右子树成为t的右子树;
s成为t的前驱;
t成为s的右孩子;
s的后继的前驱由s改为t



求中序线索树中p的后继q:
p->rtag=1时, $q=p \rightarrow rc$;
p->rtag=0时, q是p的右子树的最左结点



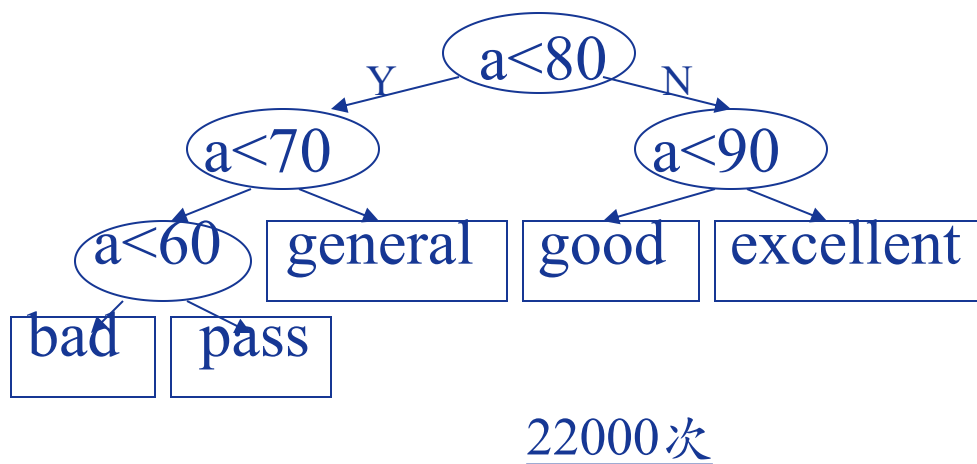
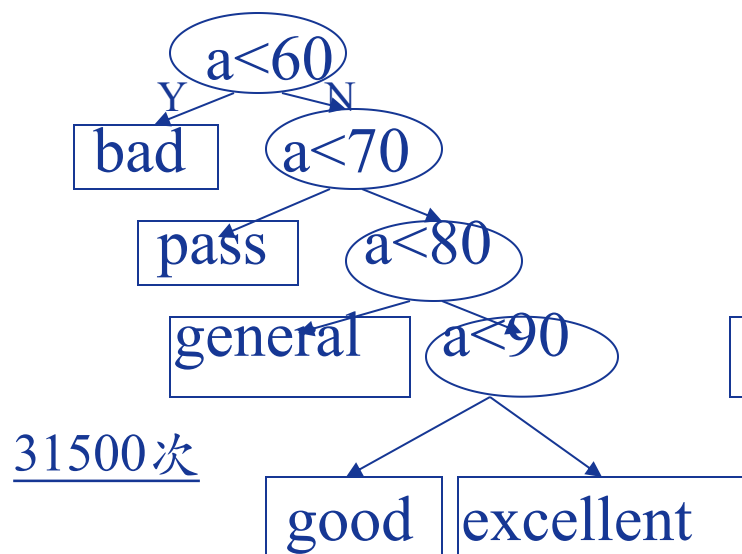
6.7 哈夫曼树

BUPT

哈夫曼树（最优二叉树）的概念

[例] 编制将百分制转换为五级分制的程序(10000数据)

分数	0—59	60—69	70—79	80—89	90—100
比例	0.05	0.15	0.40	0.30	0.10



树的路径长度 从树根到每一个结点的路径上的分支数。

带权路径长度 结点的路径长度与该结点的权之积。

树的带权路径长度 树中所有叶子结点的带权路径长度之和。

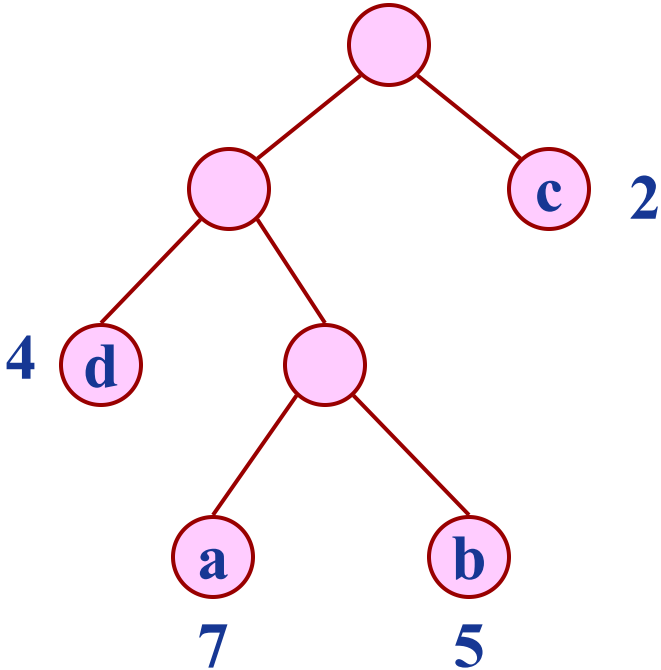
$$\text{记作: } wpl = \sum_{k=1}^n w_k l_k$$

其中: w_k —— 权值

l_k —— 结点到根的路径长度

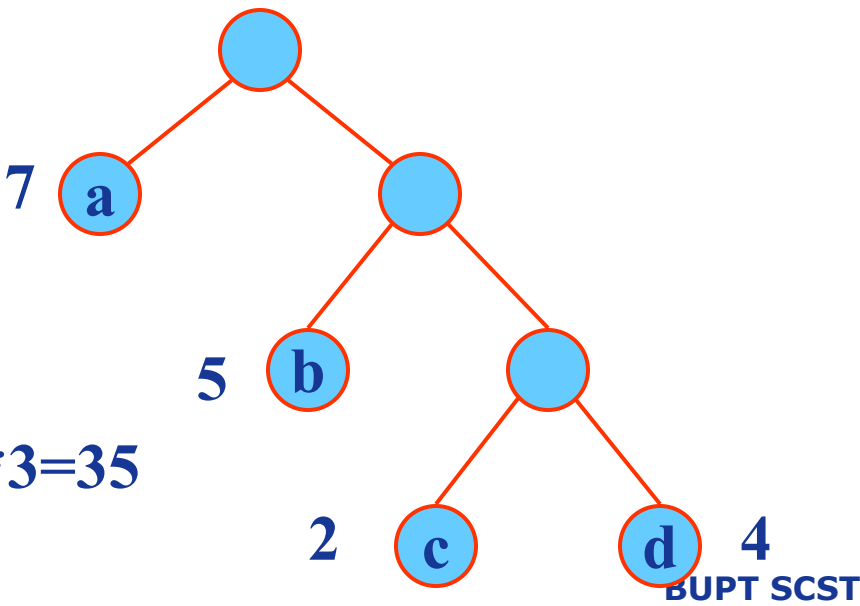
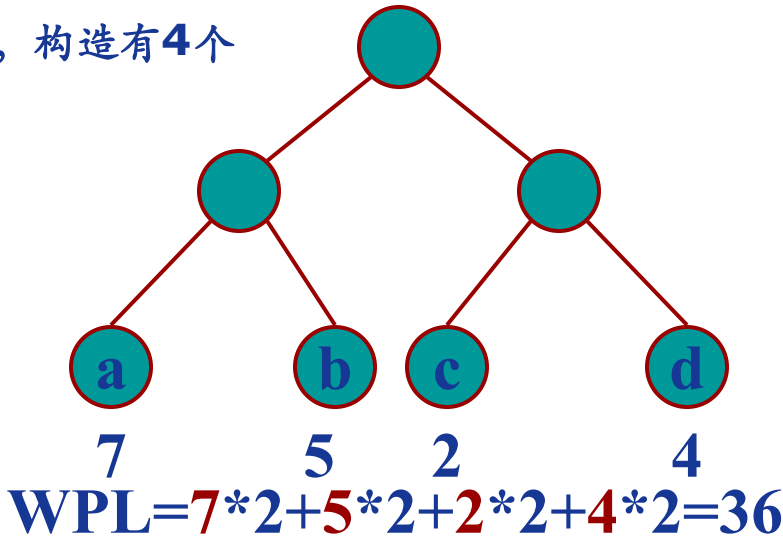
最优二叉树(哈夫曼树) 带权路径长度WPL最小的二叉树。

[例] 有4个结点，权值分别为7，5，2，4，构造有4个叶子结点的二叉树



$$WPL=7*3+5*3+2*1+4*2=46$$

$$WPL=7*1+5*2+2*3+4*3=35$$



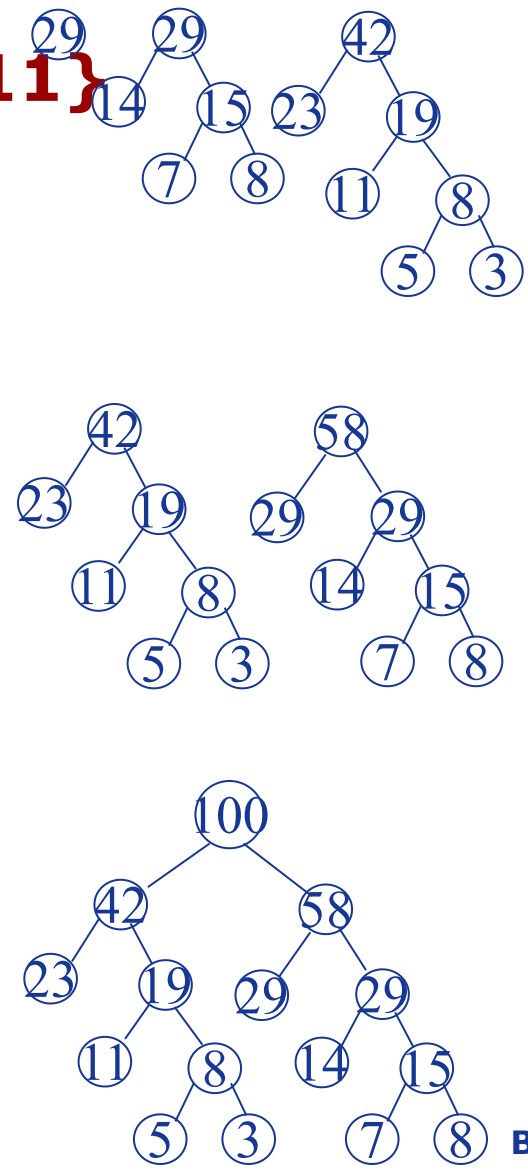
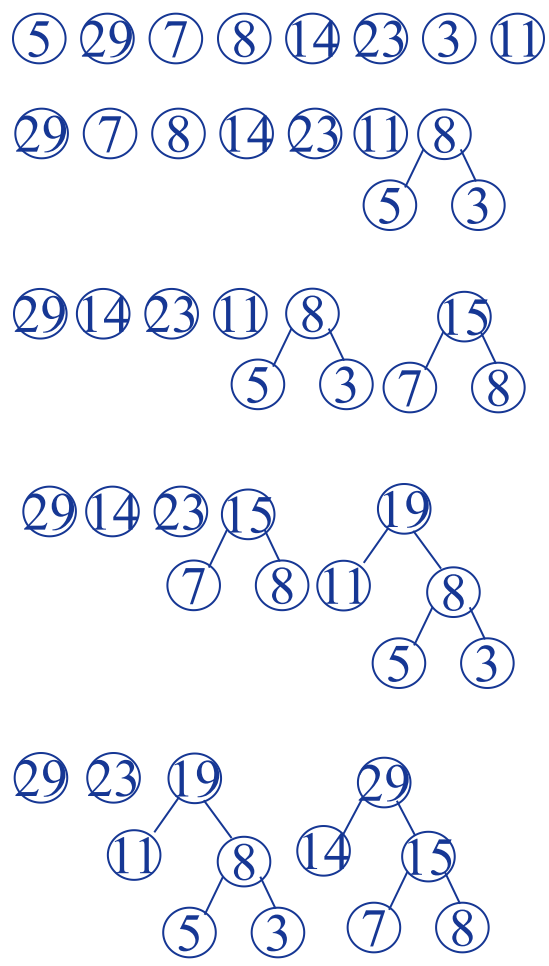
[建立哈夫曼树（最优二叉树）的方法]

[基本思想]

使权大的结点靠近根。

- (1)将给定权值从小到大排序成 $\{w_1, w_2, \dots, w_m\}$ ，生成一个森林 $F = \{T_1, T_2, \dots, T_m\}$ ，其中 T_i 是一个带权 w_i 的根结点，它的左右子树均空。
- (2)把 F 中根的权值最小的两棵二叉树 T_1 和 T_2 合并成一棵新的二叉树 T ： T 的左子树是 T_1 ，右子树是 T_2 ， T 的根的权值是 T_1 、 T_2 树根结点权值之和。
- (3)将 T 按权值大小加入 F 中，同时从 F 中删去 T_1 和 T_2
- (4)重复(2)和(3)，直到 F 中只含一棵树为止，该树即为所求。

[例] $w=\{5, 29, 7, 8, 14, 23, 3, 11\}$

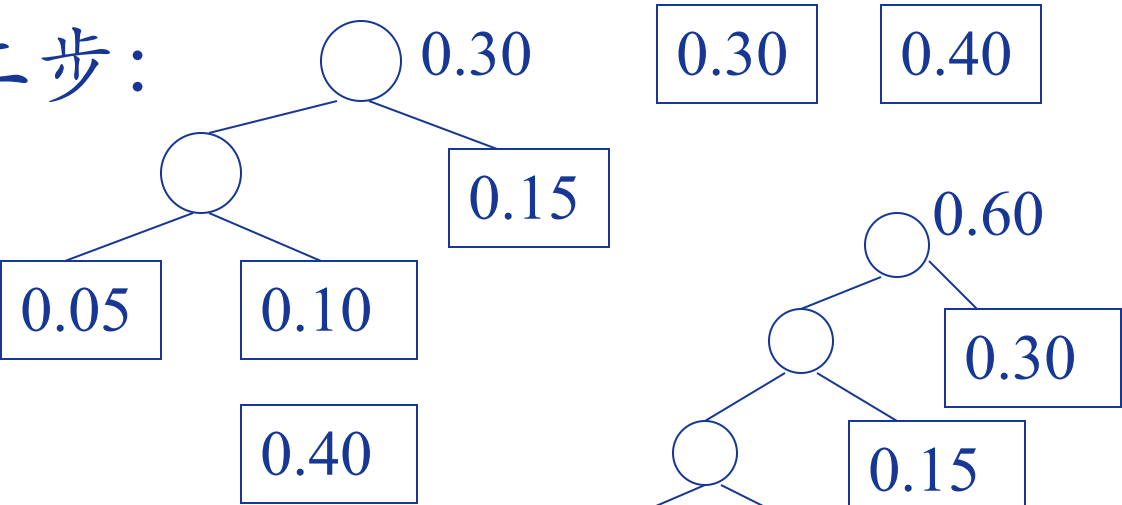




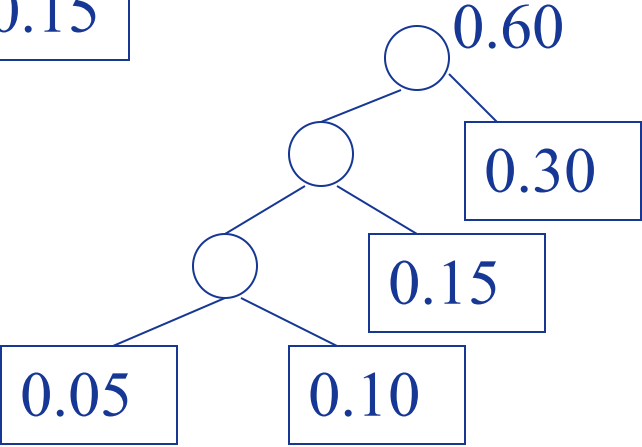
第一步:



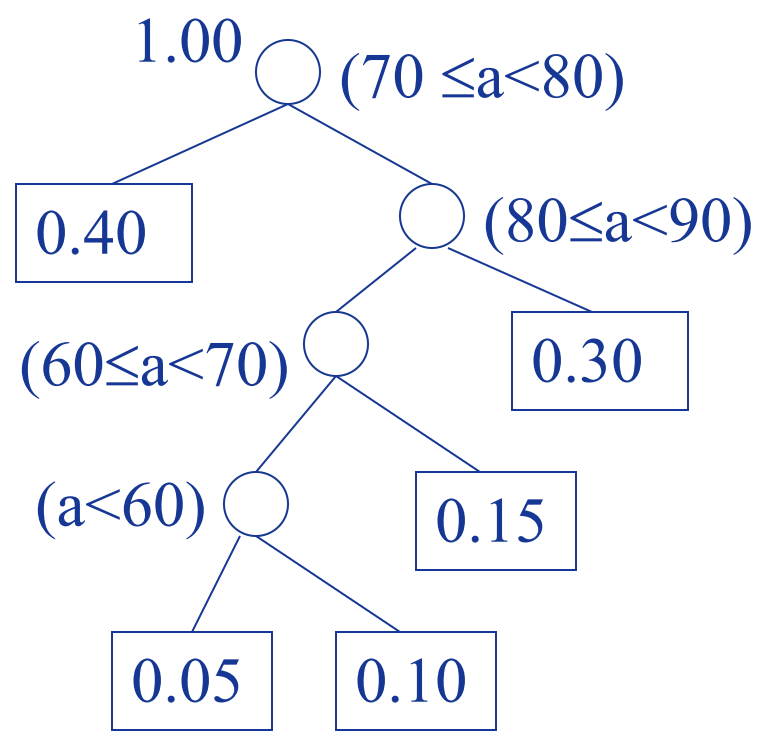
第二步:



第三步:



第四步：



20500次 (10000数据)

[哈夫曼树的存储结构]

- 初始有 n 个叶子结点，则构造出的哈夫曼树(属于严格二叉树)共有 $2n-1$ 个结点。
- 用大小为 $2n-1$ 的向量存储哈夫曼树——顺序存储。

[哈夫曼树结构]

```
struct hfmnode
{
    elemtype data;
    float weight;
    int lc;
    int rc;
    int parent;
}
typedef struct hfmnode HTNode;
HTNode ht[2n-1];
```

[哈夫曼树实现的算法步骤]

- 1)初始化 $ht[2n-1]$:
weight=0, lc=rc=parent=-1;
- 2)输入初始 n 个叶子结点: 置 $ht[0..n-1]$ 的weight;
- 3)进行以下 $n-1$ 次合并, 依次产生 $ht[i]$, $i=n..2n-2$:
 - 3.1)在 $ht[0..i-1]$ 中选两个未被选过的weight最小的两个结点 $ht[s1]$ 和 $ht[s2]$ (从parent = -1 的结点中选)
 - 3.2)修改 $ht[s1]$ 和 $ht[s2]$ 的parent值: parent= i ;
 - 3.3)置 $ht[i]$ weight= $ht[s1].weight + ht[s2].weight$,
lc= $s1$, rc= $s2$

[存储示意图]

	0	1	2	...	s1	...	s2	...	n-1	n	...	2n-2
weight	w ₀	w ₁	w ₂	...	w _{s1}	...	w _{s2}	...	w _j			
lc	-1	-1	-1	...	-1	...	-1	...	-1	-1	-1	-1
rc	-1	-1	-1	...	-1	...	-1	...	-1	-1	-1	-1
parent	-1	-1	-1	...	-1	...	-1	...	-1	-1	-1	-1

初始状态

	0	1	2	...	s1	...	s2	...	n-1	n	...	2n-2
weight	w ₀	w ₁	w ₂	...	w _{s1}	...	w _{s2}	...	w _{n-1}	w _{s1} +w _{s2}		
lc	-1	-1	-1	...	-1	...	-1	...	-1	s1	-1	-1
rc	-1	-1	-1	...	-1	...	-1	...	-1	s2	-1	-1
parent	-1	-1	-1	...	n	...	n	...	-1	-1	-1	-1

i为根和j为根的两棵树合并

[哈夫曼树的应用]

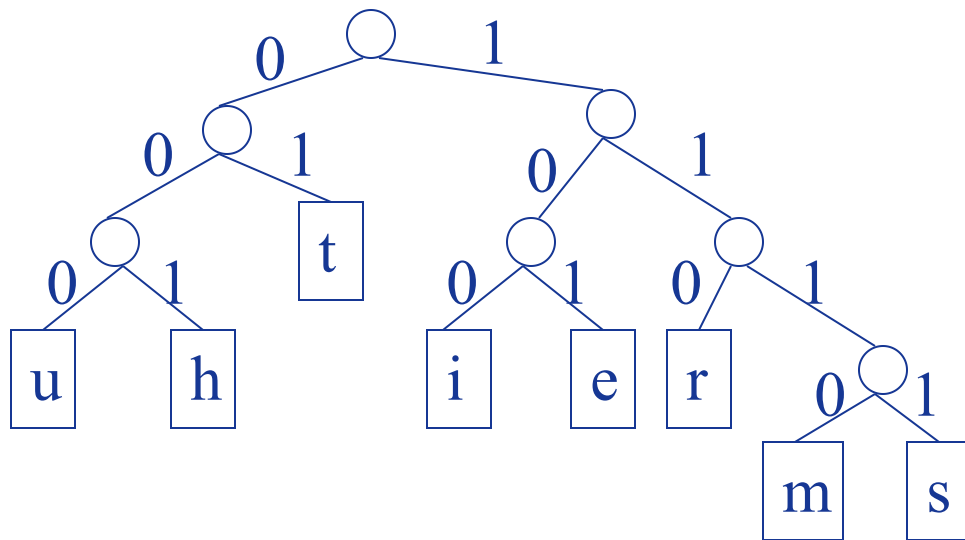
❖ 最佳判定树

❖ 哈夫曼编码：用于通信和数据传送中字符的二进制编码，可以使电文编码总长度最短。

[例] 对time tries truth哈夫曼编码

字符集 $D=\{t, i, m, e, r, s, u, h\}$

出现频次 $W=\{4, 2, 1, 2, 2, 1, 1, 1\}$



t	01
i	100
m	1110
e	101
r	110
s	1111
u	000
h	001

字符串	ABRACADABRA!
ASCII字符编码	100000110000101010010100000110000111000001- 100010010000011000010101001010000010100001. 7*12 = 84bits
Huffman编码	! 1010 A 0 B 111 C 1011 D 100 R 110 011111001011010011111001010 27bits

标准的ASCII大约有100个可打印的字符组成，大约需要 $\log_2 100 = 7$ 位二进制， $2^7=128$ ，ASCII字符还可以加上一些不可打印的字符。关键在于ASCII编码，字符集的大小是C，标准编码就需要 $\log_2 C$ 位。

在现实中，文件可能相当大，按ASCII编码不实际，尤其需要进行网络传输时，更加迫切希望减少文件的大小，因此降低文件总位数的编码就是一个现实的需求。

[总结]

- 哈夫曼编码是不等长编码
- 哈夫曼编码是前缀编码，即任一字符的编码都不是另一字符编码的前缀
- 哈夫曼编码树中没有度为1的结点。若叶子结点的个数为 n ，则哈夫曼编码树的结点总数为 $2n-1$
- 发送过程：根据由哈夫曼树得到的编码表送出字符数据
- 接收过程：按左0、右1的规定，从根结点走到一个叶结点，完成一个字符的译码。反复此过程，直到接收数据结束

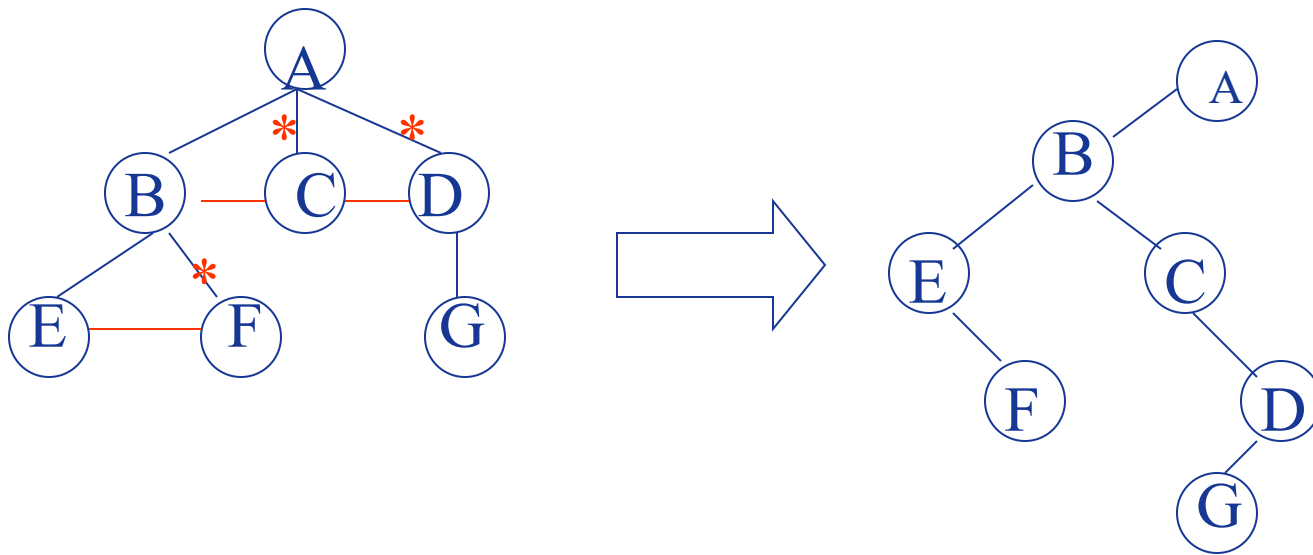
6.8 树和森林

BUPT

森林与二叉树的转换

[树转换为二叉树]

- 1) 在兄弟间加一连线
- 2) 对每一结点，去掉它与孩子的连线(最左子除外)
- 3) 以根为轴心将整棵树顺时针转 45°



特点：

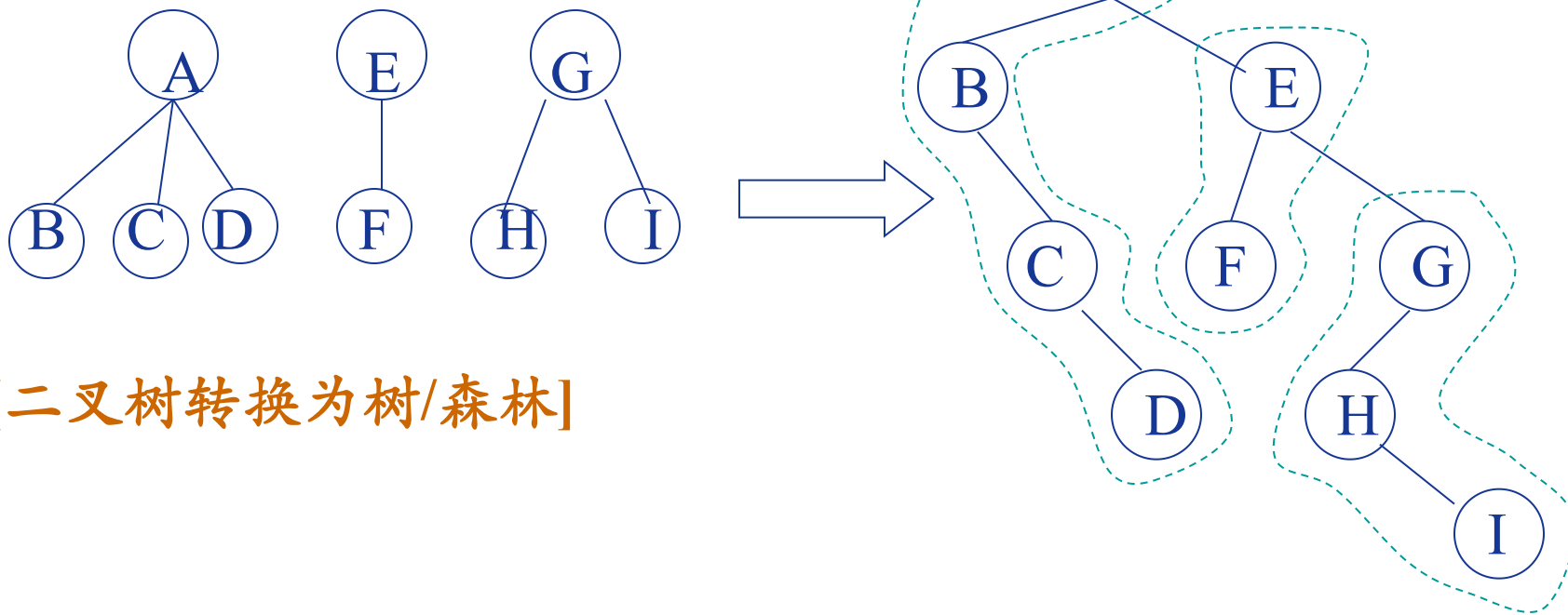
无右子树

左支是孩子

右支是兄弟

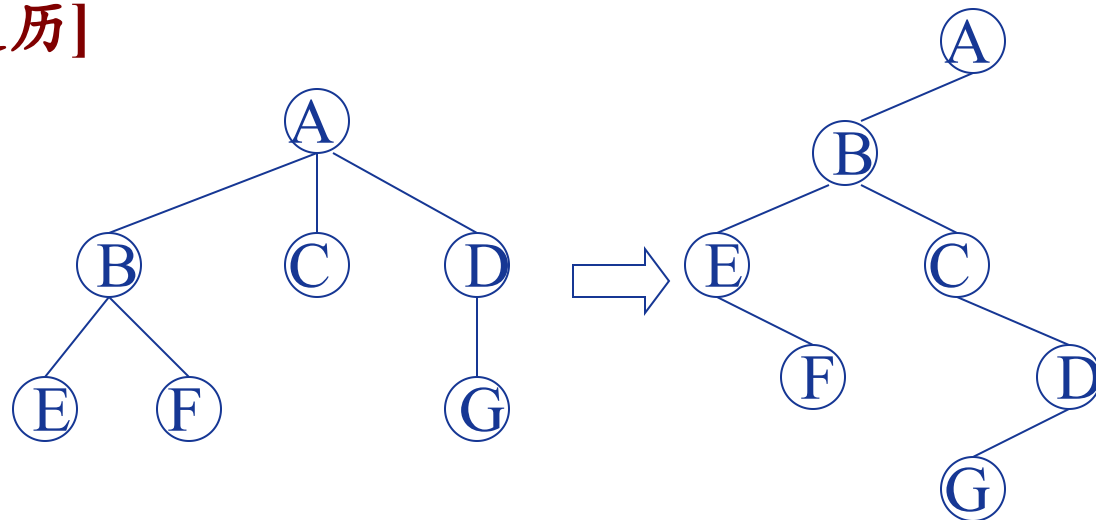
[森林转换为二叉树]

- 1) 先将森林里的每一棵树转换成一棵二叉树
- 2) 从最后一棵树开始，把后一棵树的作为前一棵树的根的右子



[二叉树转换为树/森林]

[树的遍历]



- ❖ **先序遍历** 先访问树的根结点，然后依次先根遍历根的每棵子树
ABEFCDDG(二叉树先序)
- ❖ **后序遍历** 先依次后根遍历根的每棵子树，然后访问树的根结点
EFBCGDDA(二叉树中序)

[森林的遍历]

❖ 先序遍历

访问第一棵树的根结点;
先序遍历第一棵树的根
的子树森林;
先序遍历除第一棵树外
剩余的树构成的森林

(逐棵先序遍历每棵子树/
对应二叉树的**先序遍历**)

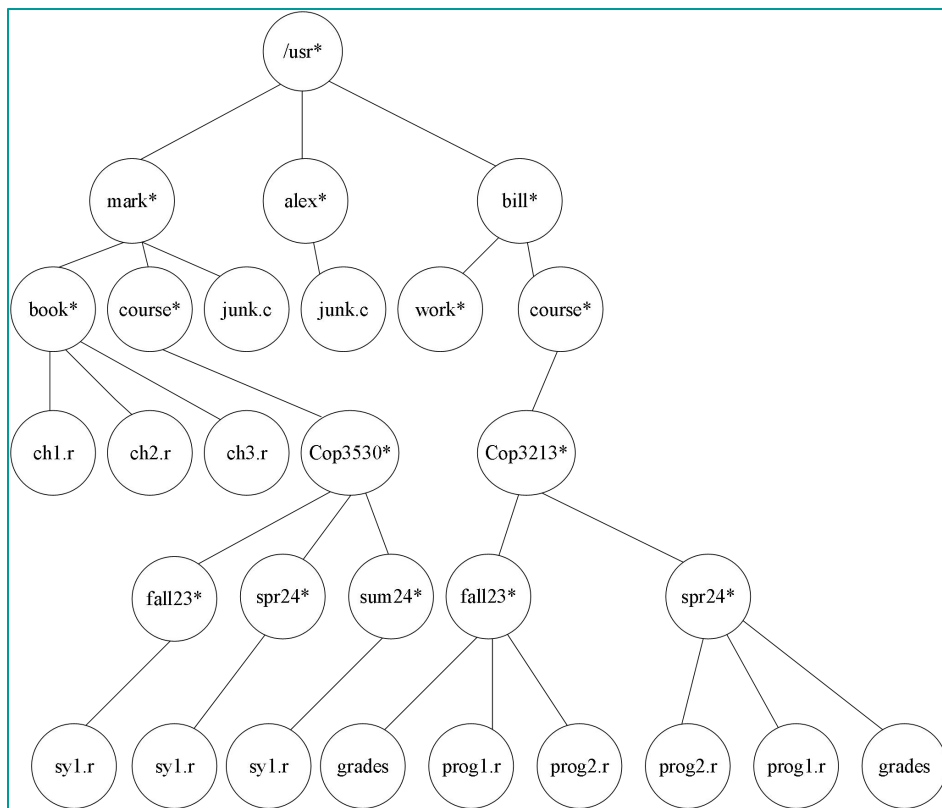
❖ 中序遍历

中序遍历第一棵树的根
的子树森林;
访问第一棵树的根结点;
中序遍历除第一棵树外
剩余的树构成的森林

(逐棵后序遍历每棵子树/
对应二叉树的**中序遍历**)

[树遍历的应用]

Unix文件系统的目录结构-典型的树结构。



[特点]

- 带*的为目录，不带*的为文件
- 根结点：/usr；
- /usr三个儿子/mark, /alex, /bill均为目录；
- 文件名/usr/mark/book/ch1.r是全路径名，可通过三次求最左子得到，除第一/外，其他每一个/都代表一个分支；
- 不同目录下文件名可以相同。

算法1： 输出目录中所有文件的名字，深度为 d_i 的文件用 d_i 次空格缩进打印，根结点不缩进（假设根在第0层）。

```

/usr
mark
  book
    ch1.r
    ch2.r
    ch3.r
  course
    cop3530
    fall23
      syl.r
    spr24
      syl.r
    sum24
      syl.r
  junk.c
alex
  junk.c
bill
  work
.....

```

【算法步骤】

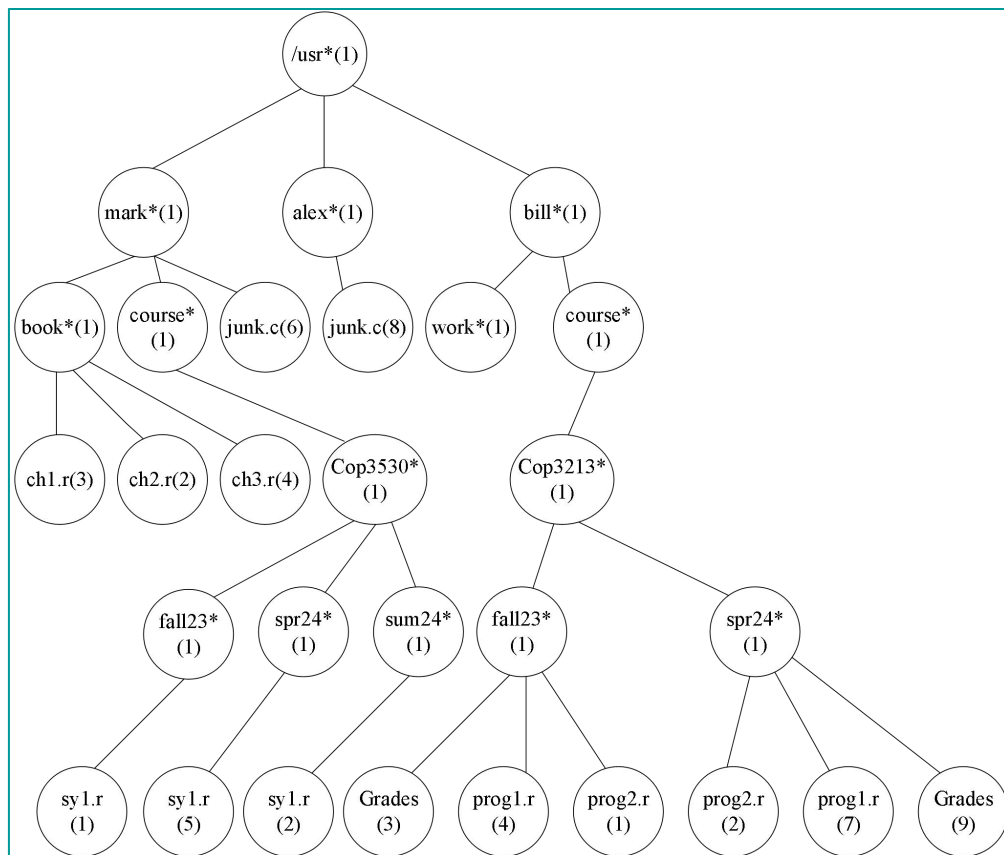
```

static void ListDir(DirectoryofFile D, int Depth)\\树先序
{
  if (D is a legitimate entry)
  {
    PrintName(D, Depth);\\按深度Depth打印文件名
    if (D is a directory)
      for (each child C of D)
        ListDir(C, Depth+1);
  }
}

Void ListDirectory (DirectoryofFile D)
{
  ListDir(D,0);\\根结点不缩进
}

```

算法2： 文件名后括号中标注了每个文件名所占磁盘区块数，要求计算所有文件占用的磁盘区块总数。



思路：

先算出/usr/mark(30);/usr/alex(9);/usr/bill(32);

子目录的总块数=30+9+32=71;

/usr本身占用的1块;

得出/usr(72)

采用树的后序遍历

[算法步骤]

```
static int SizeDirectory(DirectorofFile D)
```

```
{//树的后序
```

```
int TotalSize = 0;
```

```
if (D is legitimate entry)
```

```
{
```

```
    TotalSize = FileSize(D);
```

```
    if (D is Directory)
```

```
        for (each child C of D)
```

```
            TotalSize +=SizeDirectory(C);
```

```
}
```

```
return TotalSize;
```

```
}
```

[树的存储结构]

[多重链表表示法]

结点结构

data	child ₁	child ₂	...	child _m
------	--------------------	--------------------	-----	--------------------

数据域 指针域 (m: 树的度)

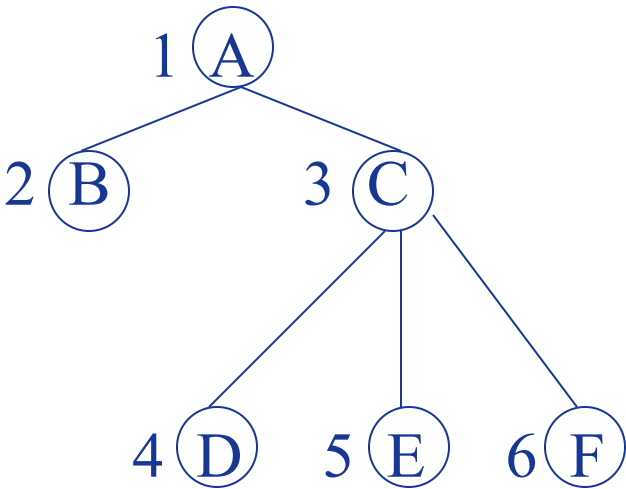
[双亲表示法] 用一维数组存储树的信息

结点序号

1
2
3
4
5
6

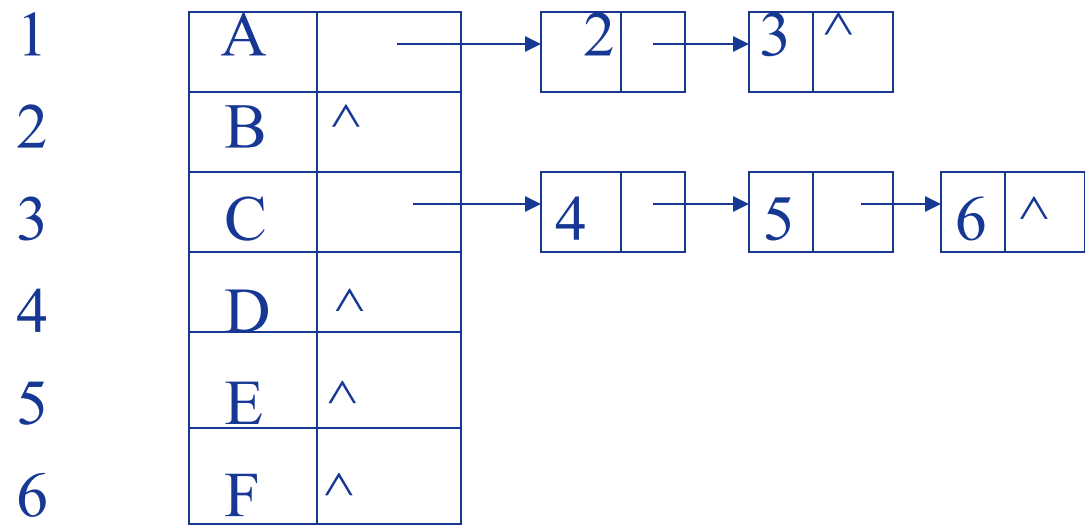
data parent

A	0
B	1
C	1
D	3
E	3
F	3



[孩子链表表示法] 数组+单链表

结点序号 data next



[孩子兄弟表示法(二叉链表表示法)]

结点结构

firstchild	data	nextsibling
------------	------	-------------

firstchild: 指向该结点的第一个孩子结点

nextsibling: 指向该结点的下一个兄弟结点

双亲表示法的定义

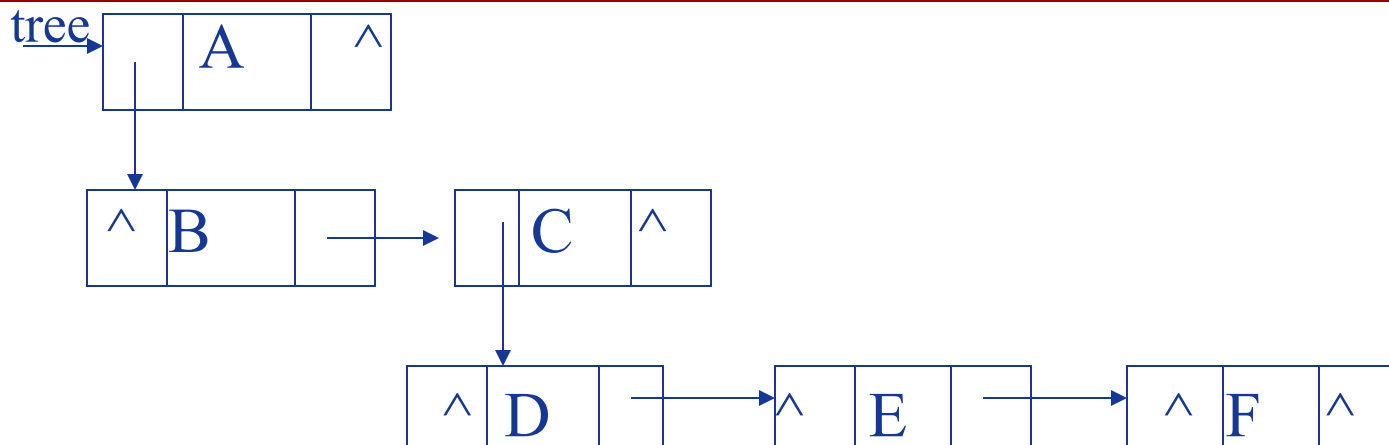
```
struct ptree
{
    elemtype data;
    int parent;
}
typedef struct ptree Ptree;
```

孩子兄弟链表的定义

```
struct TreeNode
{
    elemtype data;
    struct TreeNode *firstchild;
    struct TreeNode *nextsibling;
}
typedef struct TreeNode * CSTree;
```

孩子链表的定义

```
typedef struct node { // 孩子链表结点
    int child; // 孩子下标
    struct node *next;
} Node;
typedef struct treenode { // 树结点
    char data; // 树结点值
    Node *next; // 指向第一个孩子的指针
} Treenode;
typedef struct { // 孩子链表表示的树
    Treenode td[maxn]; // 结点数组
    int root; // 树根结点下标, 空树为-1
} Ctree;
```



孩子兄弟链表访问某个结点的所有孩子结点：

从firstchild链找到第一个孩子，沿第一个孩子的nextsibling链找第二个孩子，……，直至nextsibling链为空。

6.8 并查集

BUPT

[等价类]

假设集合 S 中元素为 $0, 1, 2, 3, \dots, n-1$ 。 $S=S_1 \cup \dots \cup S_k$ ，且当 $i \neq j$ 时，有 $S_i \cap S_j = \emptyset$ ，则称集合 $S_1 \dots S_k$ 为**等价类**。

[实例1]

当 $n = 10$ ，元素可划分为三个互不相交的集合（等价类）：

$S_1 = \{0, 6, 7, 8\}$ ， $S_2 = \{1, 4, 9\}$ 和 $S_3 = \{2, 3, 5\}$ 。

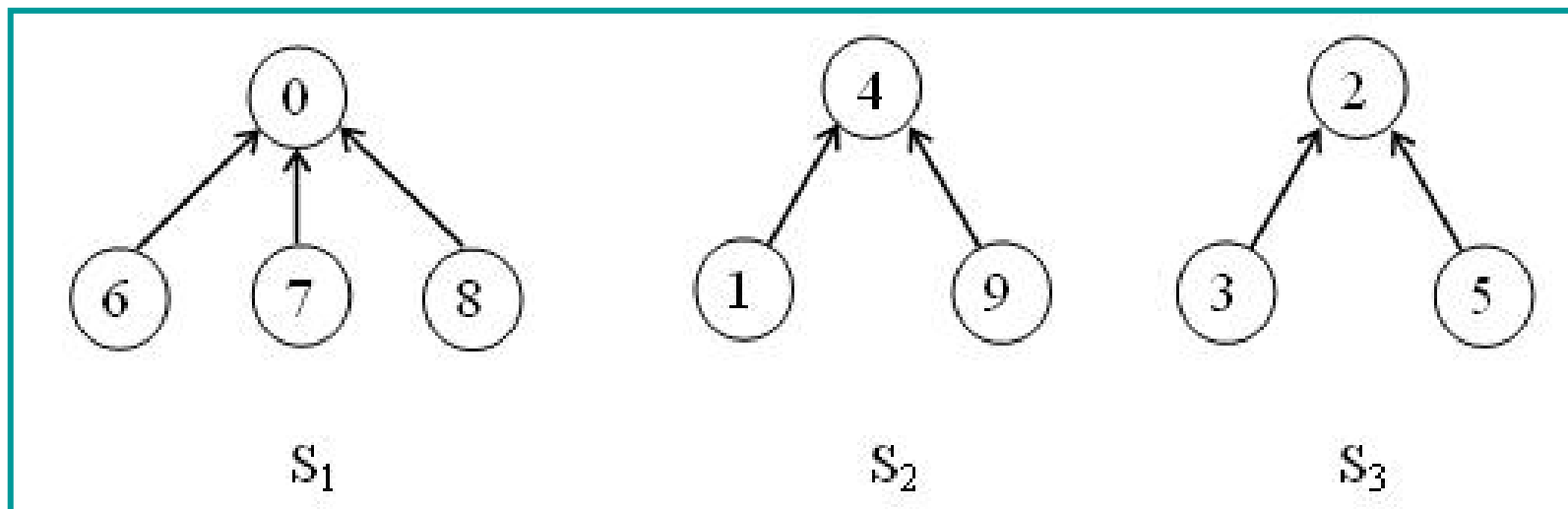
[并查集]

即在等价类基础上进行如下操作：

- $\text{merge}(S_i, S_j)$ ：求 S_i 和 S_j 的**并**（ $\cup$ ）。 S_i 和 S_j 不再独立存在，将被 $S_i \cup S_j$ 所取代。
- $\text{find}(i)$ ：**查**找含元素 i 的集合。

为了高效率实现并和查的操作，等价类可以用表示为**树**的集合（即**森林**）。

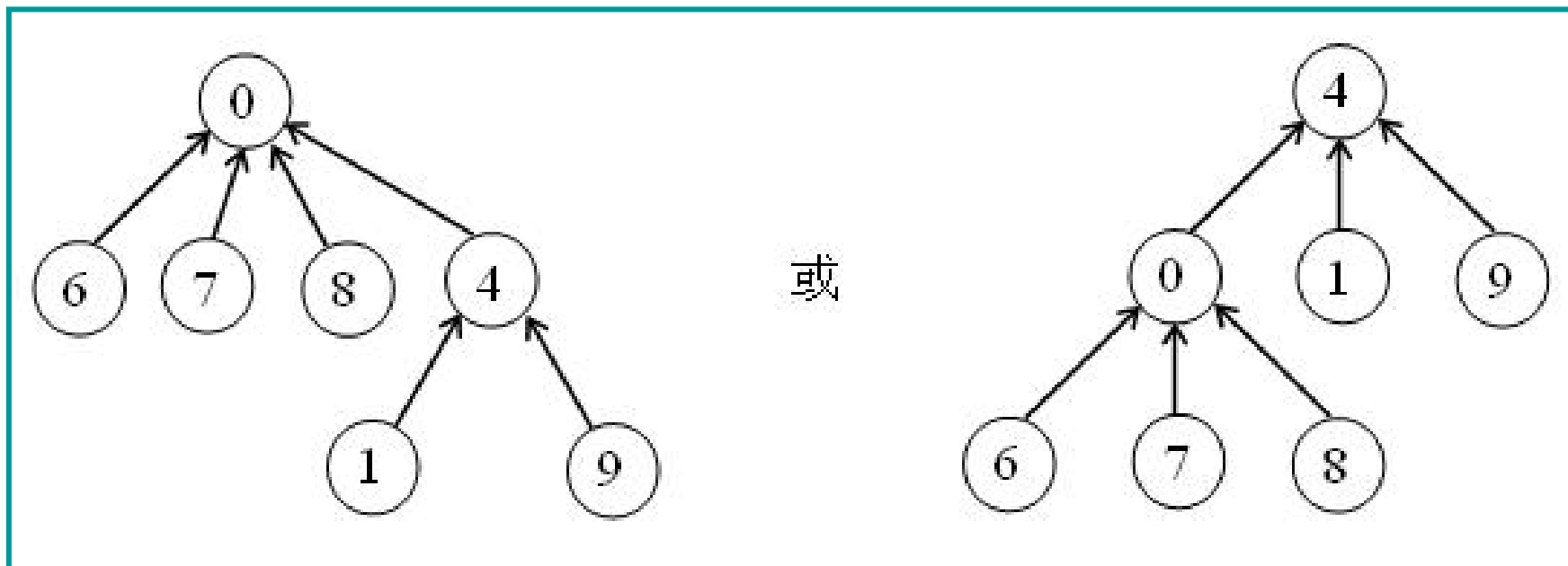
如，实例1中的 S_1 ， S_2 和 S_3 的表示如下：



其中，等价类中的每个集合用一棵树表示，且**分枝**由子结点指向双亲。

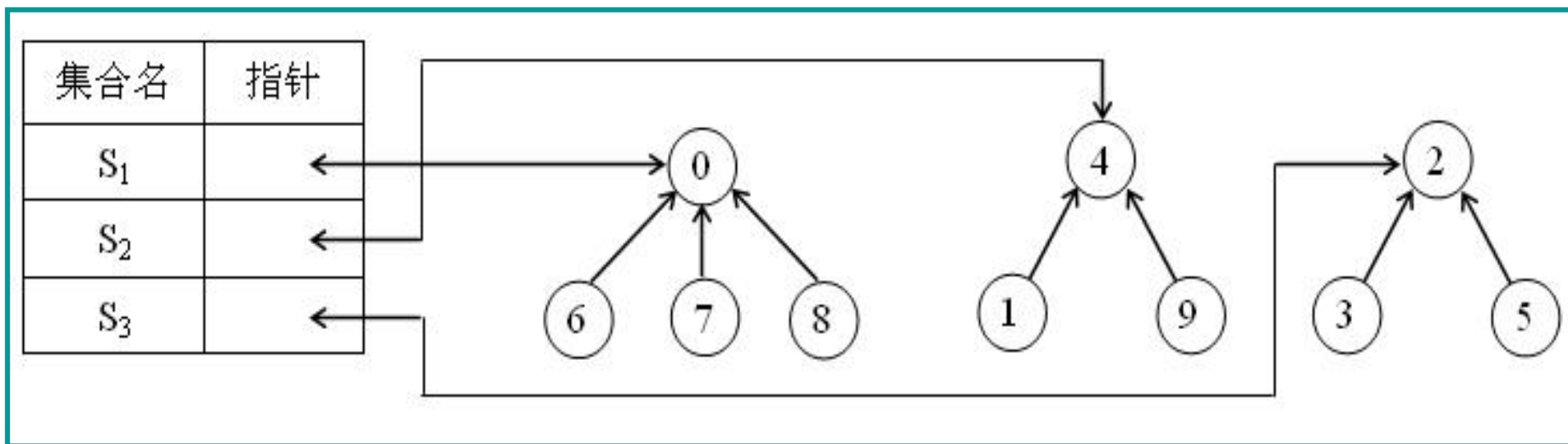
实现并操作可直接使两棵树之一成为另一棵树的子树。

如 $S_1 \cup S_2$ 可表示为下列两种形式之一：



如果每个集合名对应一个指针，使之指向表示该集合的树的根，则并操作很容易完成。

此外，每个根也存放一个指向集合名的指针，这样，当需要判断一个元素属于哪个集合时，可沿着子结点到双亲的链接，定位于根，并确定集合名。



以后将直接用树根表示集合。于是，查操作 $\text{find}(i)$ 变为：确定含元素 i 的树的根；并操作 $\text{merge}(i, j)$ 变为：需要连接以 i 为根和以 j 为根的两棵树。

由于集合元素为 $0, 1, 2, 3, \dots, n-1$ ，可以用数组`parent[Max]`表示树结点。数组的第 i 个位置表示元素 i 对应的树结点。数组元素存放相应树结点的双亲指针。

则实例1中 S_1 ， S_2 和 S_3 的数组表示如下：

***注意根结点的parent为-1。

i	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
parent	-1	4	-1	2	-1	2	0	0	0	4

基于上述表示的并查集的定义和构造函数：

```
class Sets {  
public:  
    // 集合操作  
    ...  
private:  
    int *parent;  
    int n;        // 集合元素个数  
};  
  
Sets::Sets (int sz = DefaultSize) {  
    n = sz;  
    parent = new int[sz];  
    for (int i = 0; i < n; i++) parent[i] = -1;  
}
```

查操作只需简单地沿着结点*i*的parent向上移动，直至到达parent值为-1的结点：

```
int Sets::SimpleFind (int i) {           // 找到含元素i的树的根
    while (parent[i] >= 0) i = parent[i];
    return i;
}
```

并操作也很简单，这里假设采用令第一棵树为第二棵的子树的策略：

```
void Sets::SimpleMerge (int i, int j) { // 用以i 为根和以j
    // (i!=j) 为根的两个不相交集合并取代它们
    parent[i] = j;
}
```

[SimpleMerge和SimpleFind的复杂度分析]

算法简单，但性能差。

例，假设一开始各集合只含一个元素，即 $S_i = \{i\}$, $0 \leq i < n$ 。

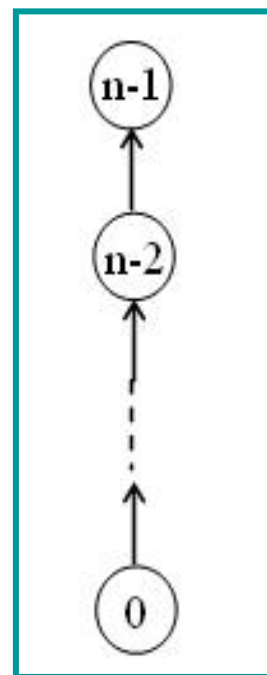
执行以下操作序列：

$\text{merge}(0,1), \text{merge}(1,2), \text{merge}(2,3), \dots, \text{merge}(n-2,n-1),$
 $\text{find}(0), \text{find}(1), \dots, \text{find}(n-1)$

将生成一颗的**退化树**。

该序列中的 $n-1$ 个 merge 共需 $O(n)$ 时间。然而，每个 find 都需要从相应元素所在结点出发，沿 parent 指针找到根。从位于第 i 层的结点找到根结点需要 $O(i)$ 时间，完成 n 个 find

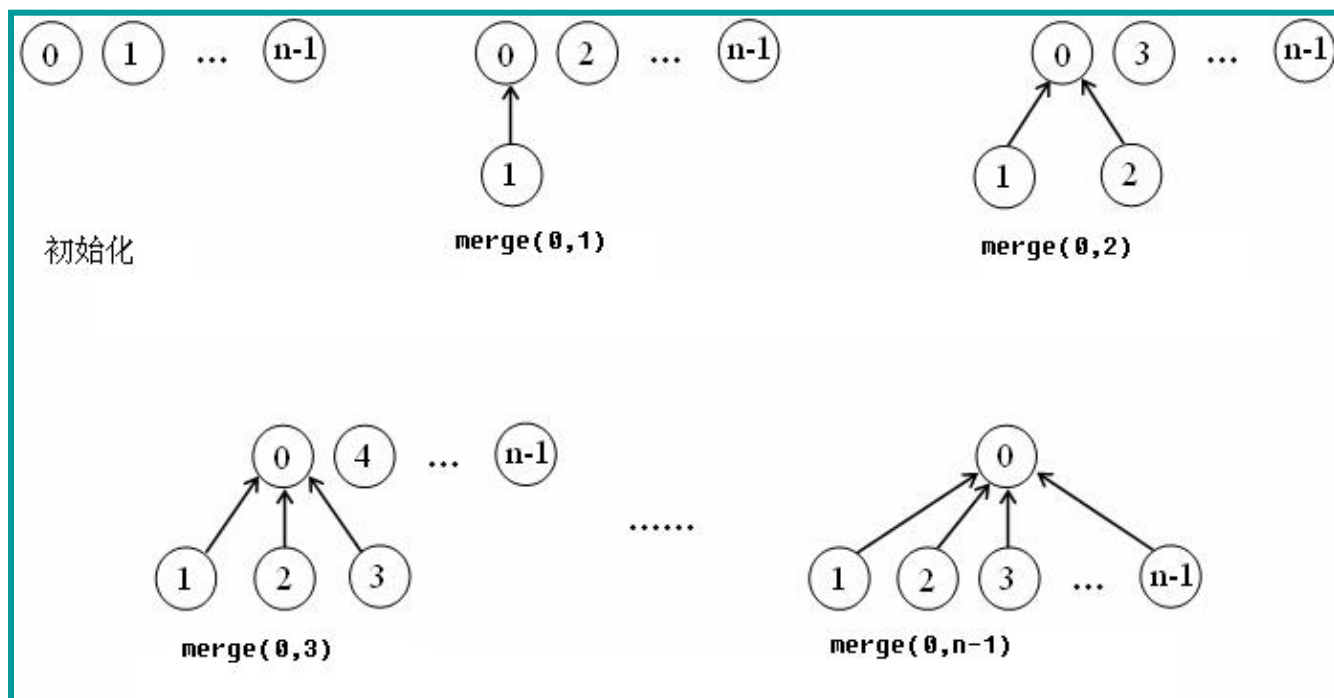
共需 $O(\sum_{i=1}^n i) = O(n^2)$ 时间。



[加权合并]

可以通过**避免生成退化树**来改善merge和find操作的性能。

- ❖ 对merge(i, j)操作应用以下**加权规则**：若以 i 为根的树中结点数小于以 j 为根的树中结点数，则令 j 成为 i 的双亲，否则令 i 成为 j 的双亲。



[加权合并的实现]

利用根结点的parent字段以负数的形式存储计数数据。

```
void Sets::WeightedMerge (int i, int j) {  
    // 基于权重规则构造以i和j 为根的集合的并  
    int temp = parent[i] + parent[j];  
    if ( parent[j] < parent[i] ) {           // 树i 的结点少  
        parent[i] = j; parent[j] = temp;  
    }  
    else {                                   // 树j 的结点少或与树i的同样多  
        parent[j] = i; parent[i] = temp;  
    }  
}
```

[WeightedMerge和SimpleFind的复杂度分析]

一次merge操作的时间仍然是 $O(1)$ 。find操作未变，其一次执行所需要的最长时间可由以下定理6.8确定。

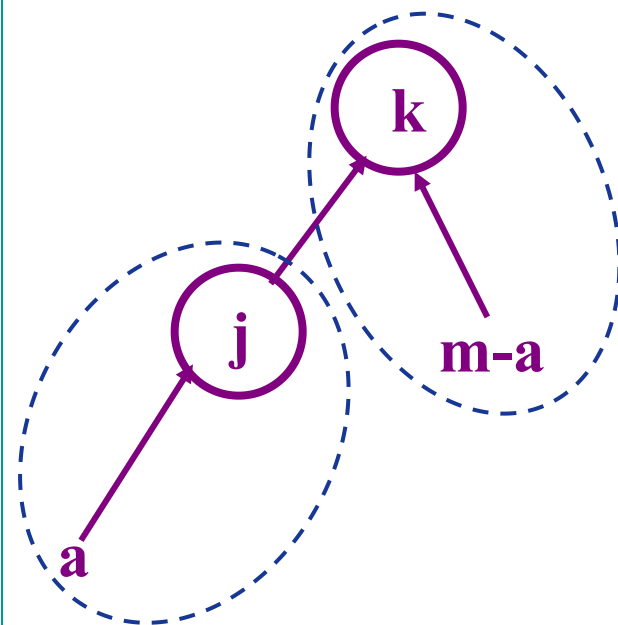
定理：假设开始时森林是由 n 个含单个结点的树构成的，通过任意次的WeightedMerge操作，森林中的任一子树高度不高于 $\lfloor \log_2 n \rfloor + 1$ 。

引理：假设开始时森林是由含单个结点的树构成的，并令 T 为一棵具有 m 个结点，且通过执行一系列WeightedMerge函数构成的树，则 T 的高度不高于 $\lfloor \log_2 m \rfloor + 1$ 。

Proof: 当 $m = 1$ ，显然正确。假设对于所有具有 i ($i \leq m - 1$) 个结点的树引理正确。下面需要证明对于 $i = m$ 引理也正确。

设 T 是由函数WeightedMerge构建的，有 m 个结点的树。考虑最后一次并操作WeightedMerge(k, j)。设 a 为树 j 的结点数，那么树 k 的结点数为 $m - a$ 。不失一般性，设 $1 \leq a \leq m/2$ ，则 T 的根为 k 。

因此， T 的高度或者与原树 k 的相同或者比原树 j 的多一级。若是前一种情况， T 的高度 $\leq \lfloor \log_2 (m - a) \rfloor + 1 \leq \lfloor \log_2 m \rfloor + 1$ ；若是后一种情况， T 的高度 $\leq \lfloor \log_2 a \rfloor + 2 \leq \lfloor \log_2 m \rfloor + 1$ 。



[结论]

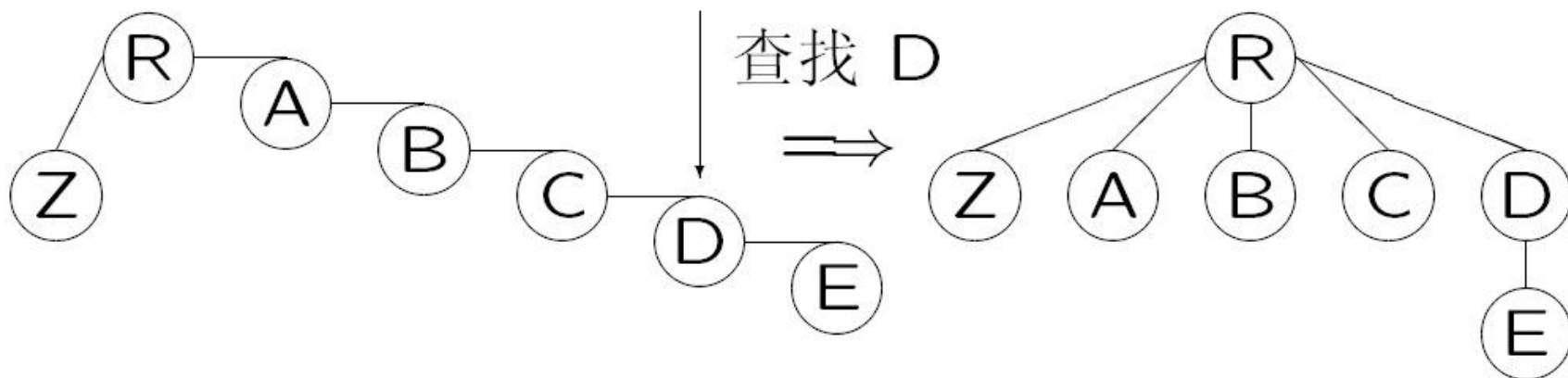
由Th6.8知：如果一棵树有 n 个结点，则一次find操作的时间最多为 $O(\log n)$ 。

执行 $u-1$ 次merge操作和 f 次find操作组成的混合序列的分摊复杂度是 $O(u + f \log u)$ ，因为每棵树的结点数不超过 u 。

初始化 n 棵树的森林需要 $O(n)$ 时间。

[折叠查找]

问题的解决方法还可以作进一步的改进：这次将对find操作应用以下**坍塌规则**：将被查结点到其根的路径上所有结点嫁接到根上，即：若j是从i到根的路径中的结点，且 $\text{parent}[j] \neq \text{root}(i)$ ，则将 $\text{parent}[j]$ 设置为 $\text{root}(i)$ 。



[折叠查找的实现]

```
int Sets::CollapsingFind (int i ) {    // 找含元素i的树的根,  
    for (int r = i; parent[r] >= 0; r = parent[r]); // 找到根  
    while ( i != r )  
    {  
        int s = parent[i];  
        parent[i] = r;  
        i = s;  
    }  
    return r;  
}
```

[WeightedMerge和CollapsingFind的复杂度分析]

假设开始时森林是由一组只含单个结点的树构成的，且 $u \geq n/2$ ，并令 $T(f, u)$ 为执行 u 次WeightedMerge操作和 f 次CollapsingFind操作组成的混合序列的最长时间，那么对于正常数 k_1 和 k_2 ，有

$$k_1(n + f \alpha(f + n, n)) \leq T(f, u) \leq k_2(n + f \alpha(f + n, n))$$

$\alpha(p, q)$ 函数增长非常缓慢，其定义为：

$$\alpha(p, q) = \min\{z \geq 1 \mid A(z, \lfloor p/q \rfloor) > \log_2 q\}, \quad p \geq q \geq 1$$

其中，Ackerman函数 $A(i, j)$ 定义为：

$$A(1, j) = 2j \quad \text{当 } j \geq 1$$

$$A(i, 1) = A(i - 1, 2) \quad \text{当 } i \geq 2$$

$$A(i, j) = A(i - 1, A(i, j - 1)) \quad \text{当 } i, j \geq 2$$

对于任何实际可行的 p 和 q ， $\alpha(p, q) \leq 4$ 。

$A(i, j)$ 增长非常迅速, 例如,

$$A(1, 2) = 2^2$$

$$\begin{aligned} A(3, 1) &= A(2, 2) = A(1, A(2, 1)) = A(1, A(1, 2)) = 2^{A(1, 2)} \\ &= 2^{2^2} = 16 \end{aligned}$$

$$\begin{aligned} A(4, 1) &= A(3, 2) = A(2, A(3, 1)) = A(2, 16) \\ &= A(1, A(2, 15)) = 2^{A(2, 15)} \\ &= 2^{2^{A(2, 14)}} = \dots = 2^{2^{2^{\dots^2}}} \quad \left. \vphantom{2^{2^{2^{\dots^2}}}} \right\} 16 \uparrow \end{aligned}$$

$A(3, 1) = 16$ ，当 $q < 2^{16}$ 且 $p \geq q$ ， $\alpha(p, q) \leq 3$ 。而 $A(4, 1)$ 是一个非常大的数，且在本应用中， q 是集合元素个数 n ， p 是 $n + f$ （ f 是 find 操作的次数），因此，对于任何实际可行的 p 和 q ， $\alpha(p, q) \leq 4$ 。

虽然函数 $\alpha(f, u)$ 增长非常缓慢，但求解并查集问题的时间复杂性仍然不是并和查操作次数的线性函数。

在空间方面，每个元素需要一个结点。

作业

BUPT

1. 设某二叉树的先序遍历序列为:ABCDEFGH, 中序遍历序列为:BCAEDGHFI:
(1) 试画出该二叉树对应的后序线索二叉树;
(2) 将(1)所得的二叉树转化为对应的树或森林;

2. 试编写算法, 判断两颗棵以二叉链表表示的二叉树p和q是否相似。

bool Similar(BiTree p, BiTree q)//若相似函数返回TRUE, 否则为FLASE

3. 编写算法判断一棵以二叉链表表示的二叉树是否为完全二叉树。

bool JudgeComplete(BiTree root) //root为根

二叉链表结构见课件中的定义

4. 设计算法求孩子兄弟链表表示的树的度。

int degree(CSTree root) //root为根

5. 试编写算法, 对一棵以孩子-兄弟链表表示的树统计叶子的个数。

int Count(CSTree root) //root 为根

孩子兄弟链表结构见课件中的定义

6. 一个深度为 L 的满 K 叉树有以下性质：第 L 层上的结点都是叶子结点，其余各层上每个结点都有 K 棵非空子树，如果按层次顺序从1开始对全部结点进行编号，求：

- 1) 各层的结点的数目是多少？
 - 2) 编号为 n 的结点的双亲结点（若存在）的编号是多少？
 - 3) 编号为 n 的结点的第 i 个孩子结点（若存在）的编号是多少？
 - 4) 编号为 n 的结点有右兄弟的条件是什么？如果有，其右兄弟的编号是多少？
- 请给出计算和推导过程。

7. 试证明：同一棵二叉树的所有叶子结点，在先序序列、中序序列以及后序序列中都按相同的相对位置出现（即先后顺序相同），例如先序 $\underline{a}bc$ ，后序 $\underline{b}ca$ 中序 $\underline{b}a\underline{c}$ 。

8.试给出下列有关并查集的操作序列的运算结果:

merge(1,2), merge(3,4), merge(3,5), merge(1,7),
merge(3,6), merge(8,9), merge(1,8), merge(3,10), merge(3,11),
merge(3,12), merge(3,13),
merge(14,15), merge(16,0), merge(14,16), merge(1,3),
merge(1,14).

要求

- (1) 对于merge(i,j),以i作为j的双亲;
- (2) 按i和j为根的树的高度实现merge(i,j),高度大者为高度小者的双亲;
- (3) 按i和j为根的树的结点数实现merge(i,j),结点数大者为结点数小者的双亲。

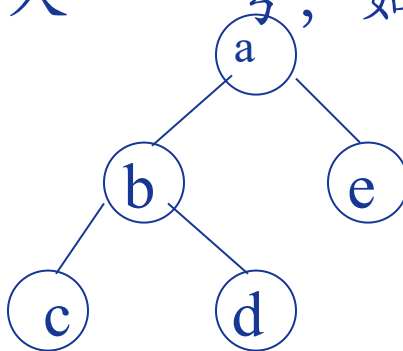
9. 设T是一棵二叉树，除叶子结点外，其它结点的度数皆为2，若T中有6个叶结点，试问：
- (1) T树的最大深度 $K_{\max}$ =? 最小可能深度 $K_{\min}$ =?
 - (2) T树中共有多少非叶结点?
 - (3) 若叶结点的权值分别为1,2,3,4,5,6。请构造一棵哈曼夫树，并计算该哈曼夫树的带权路径长度wpl。

实验

BUPT

- 1) 用先序递归过程建立二叉树(存储结构: 二叉链表), 要求: 输出二叉树的四种表示法之一(参见本章课件P6)。

输入数据按先序遍历所得序列输入, 当某结点左子树或右子树为空时, 输入 '*' 号, 如输入abc**d**e**得到的二叉树为:



2. 将任意一个指定的文件进行哈夫曼编码, 并以真正的二进制位生成一个二进制文件(压缩文件); 反过来, 可将一个压缩文件解码还原为原来的文件。

提示: 对选定的文件以二进制方式读入, 一次读入一定的长度如1024 bytes, 然后对每一字节(一字节8 bit, 所以刚好对应256个ASCII字符)进行统计, 如此循环统计